

计算机图形学 实验报告

姓 名： 张致远

学 号： 2023213980

专业班级： 信计 23-1

指导教师： 霍 星

完成日期： 2026 年 5 月 9 日

目录

1 实验一 直线的生成	5
1.1 实验目的	5
1.2 实验要求	5
1.3 关键算法及实现原理	5
1.3.1 DDA（数字微分分析器）算法	5
1.3.2 中点画线算法	6
1.3.3 Bresenham 算法（重点）	6
1.3.4 三种算法对比	7
1.4 程序调试中的问题	7
1.5 程序运行结果或数据	7
1.6 实验收获及体会	8
2 实验二 圆弧及椭圆弧的生成	10
2.1 实验目的	10
2.2 实验要求	10
2.3 关键算法及实现原理	10
2.3.1 圆的八分对称性	10
2.3.2 Bresenham 圆弧算法（重点）	10
2.3.3 中点画圆算法	11
2.3.4 中点画椭圆算法	11
2.4 程序调试中的问题	12
2.5 程序运行结果或数据	12
2.6 实验收获及体会	13
3 实验三 Bezier 曲线生成	15
3.1 实验目的	15
3.2 实验要求	15
3.3 关键算法及实现原理	15
3.3.1 Bezier 曲线的参数表示	15
3.3.2 二次 Bezier 曲线	16
3.3.3 三次 Bezier 曲线（重点）	16
3.3.4 de Casteljau 几何递推算法	16
3.3.5 分段三次 Bezier 与连续性条件	16
3.4 程序调试中的问题	17
3.5 程序运行结果或数据	17
3.6 实验收获及体会	18

4 实验四 Hermite 曲线生成	20
4.1 实验目的	20
4.2 实验要求	20
4.3 关键算法及实现原理	20
4.3.1 Hermite 曲线的参数表示	20
4.3.2 二次 Hermite 曲线	20
4.3.3 三次 Hermite 曲线（重点）	21
4.3.4 Hermite 与 Bezier 的等价	21
4.3.5 分段三次 Hermite 与 C^1 连续	21
4.4 程序调试中的问题	22
4.5 程序运行结果或数据	22
4.6 实验收获及体会	23
5 实验五 多边形的区域填充	25
5.1 实验目的	25
5.2 实验要求	25
5.3 关键算法及实现原理	25
5.3.1 多边形区域的两种描述方式	25
5.3.2 4-连通边界种子填充	25
5.3.3 8-连通边界种子填充	26
5.3.4 扫描线填充算法（重点）	26
5.3.5 两类算法对比	27
5.4 程序调试中的问题	27
5.5 程序运行结果或数据	28
5.6 实验收获及体会	28
6 实验六 二维几何变换	30
6.1 实验目的	30
6.2 实验要求	30
6.3 关键算法及实现原理	30
6.3.1 齐次坐标与统一矩阵表达	30
6.3.2 五种基本变换矩阵	30
6.3.3 复合变换的顺序	31
6.3.4 关于任意点的旋转	31
6.3.5 鼠标拾取——一点在多边形内判定	32
6.4 程序调试中的问题	32
6.5 程序运行结果或数据	32
6.6 实验收获及体会	33

7 实验七 裁剪算法	35
7.1 实验目的	35
7.2 实验要求	35
7.3 关键算法及实现原理	35
7.3.1 Cohen-Sutherland 直线裁剪（编码法）	35
7.3.2 中点分割算法	36
7.3.3 Liang-Barsky 参数法	36
7.3.4 Sutherland-Hodgman 多边形裁剪（重点）	36
7.3.5 算法对比	37
7.4 程序调试中的问题	37
7.5 程序运行结果或数据	38
7.6 实验收获及体会	38
8 实验八 三维图形变换	40
8.1 实验目的	40
8.2 实验要求	40
8.3 关键算法及实现原理	40
8.3.1 三维齐次坐标	40
8.3.2 基本变换矩阵	40
8.3.3 正等测投影（重点）	41
8.3.4 两阶段渲染管线	42
8.3.5 编程结构（与实验六一致）	42
8.4 程序调试中的问题	42
8.5 程序运行结果或数据	43
8.6 实验收获及体会	44
9 实验代码	45
9.1 实验一代码	45
9.2 实验二代码	48
9.3 实验三代码	52
9.4 实验四代码	55
9.5 实验五代码	59
9.6 实验六代码	64
9.7 实验七代码	70
9.8 实验八代码	75

1 实验一 直线的生成

1.1 实验目的

1. 理解光栅图形系统中直线段扫描转换的基本思想，体会“用离散像素逼近连续直线”这一图形学核心问题。
2. 掌握 DDA 算法、中点画线算法以及 Bresenham 算法的数学推导与实现细节。
3. 通过对三种算法的实现与比较，分析它们在效率、误差、可移植性等方面的差异，重点掌握 **Bresenham 直线生成算法** 的递推逻辑。
4. 熟悉 OpenGL 编程基本框架，能够利用 GL_POINTS 模式按算法逐像素绘制图元。

1.2 实验要求

实验环境：

- 实验设备：PC 机一台
- 操作系统：Windows 11
- 开发工具：Visual Studio 2022 + OpenGL (freeglut 3.4.0)
- 编程语言：C++

实验内容：

1. 在屏幕上分别用 DDA 算法、中点画线算法、Bresenham 算法绘制同一条直线段；
2. 不同算法采用不同颜色或线型加以区别，便于对比生成结果；
3. 通过键盘交互切换显示某一种或全部算法的输出，便于观察像素分布；
4. 重点验证 Bresenham 算法在 $0 \leq k \leq 1$ 第一象限的正确性，并扩展到任意斜率。

1.3 关键算法及实现原理

设直线段两端点为 $P_1(x_1, y_1)$ 与 $P_2(x_2, y_2)$ ，记 $\Delta x = x_2 - x_1$ ， $\Delta y = y_2 - y_1$ ，斜率 $k = \Delta y / \Delta x$ 。不失一般性，下文以 $0 \leq k \leq 1$ 、 $x_1 < x_2$ 为典型情况推导。

1.3.1 DDA（数字微分分析器）算法

DDA 利用直线的微分方程 $\frac{dy}{dx} = k$ ，沿 x 方向每步增 1，则 y 方向相应增加 k ：

$$x_{i+1} = x_i + 1, \quad y_{i+1} = y_i + k.$$

绘制时取 $\text{round}(y_{i+1})$ 作为像素纵坐标。算法直观，但需要浮点加法和取整运算，效率较低。

1.3.2 中点画线算法

将直线方程写为 $F(x, y) = ax + by + c = 0$, 其中 $a = \Delta y$, $b = -\Delta x$ 。当像素 (x_i, y_i) 已绘制后, 下一像素只可能是 $E(x_i + 1, y_i)$ 或 $NE(x_i + 1, y_i + 1)$ 。取两者中点 $M(x_i + 1, y_i + \frac{1}{2})$, 构造判别式

$$d_i = F(x_i + 1, y_i + \frac{1}{2}) = a(x_i + 1) + b(y_i + \frac{1}{2}) + c.$$

$d_i < 0$ 时中点在直线下方, 下一像素取 NE ; 否则取 E 。利用增量推导得:

$$d_{i+1} = \begin{cases} d_i + a + b, & d_i < 0 \text{ (走 } NE) \\ d_i + a, & d_i \geq 0 \text{ (走 } E) \end{cases}$$

初值 $d_1 = a + b/2$ 。为消除小数, 可整体乘 2, 即令 $D = 2d$, 全部用整数运算。

1.3.3 Bresenham 算法 (重点)

Bresenham 同样采用递推思想, 但以“误差量”代替“中点判别式”, 物理含义更直观。设当前像素为 (x_i, y_i) , 理想直线在 $x_i + 1$ 处的纵坐标为 $y_i + k$, 当累计偏离 ≥ 0.5 时纵坐标进 1。

为避免浮点运算, 令

$$P_i = 2\Delta x \cdot e_i - \Delta x,$$

则判别条件 $e_i \geq 0.5$ 等价于 $P_i \geq 0$ 。代入递推得:

$$P_{i+1} = \begin{cases} P_i + 2\Delta y - 2\Delta x, & P_i \geq 0 \\ P_i + 2\Delta y, & P_i < 0 \end{cases} \quad P_1 = 2\Delta y - \Delta x.$$

算法步骤 ($0 \leq k \leq 1$):

1. 画起点 (x_1, y_1) , 计算误差初值 $P_1 = 2\Delta y - \Delta x$, 置 $i = 1$;
2. 求下一像素 $x_{i+1} = x_i + 1$; 若 $P_i \geq 0$ 则 $y_{i+1} = y_i + 1$, 否则 $y_{i+1} = y_i$;
3. 画点 (x_{i+1}, y_{i+1}) ;
4. 更新误差: $P_i \geq 0$ 时 $P_{i+1} = P_i + 2\Delta y - 2\Delta x$, 否则 $P_{i+1} = P_i + 2\Delta y$;
5. $i = i + 1$, 若 $i < \Delta x + 1$ 转步骤 2, 否则结束。

对一般斜率的推广: 若 $|k| > 1$, 则交换 x, y 角色 (沿 y 步进); 若 $\Delta x < 0$ 或 $\Delta y < 0$, 通过 sign 控制步进方向即可, 本质算法保持不变。

1.3.4 三种算法对比

对比维度	DDA	中点画线	Bresenham
基本运算	浮点加 + 取整	整数加	整数加（仅加减）
单像素开销	较大	小	最小
误差分析	浮点累计误差	几何意义清晰	误差离散可控
推广性	易扩展到曲线	易扩展到圆/椭圆	易扩展到圆/椭圆
工程应用	教学 / 原型	较常用	硬件实现首选

表 1: 三种直线生成算法的比较

1.4 程序调试中的问题

- 斜率范围的限制问题。**最初仅按指导书给定步骤实现 $0 \leq k \leq 1$ 的情况，当输入端点斜率为负或大于 1 时输出明显错误。
 解决：在算法入口对端点进行预处理——根据 $|\Delta x|$ 与 $|\Delta y|$ 的大小决定主步进方向，并通过 $\text{sign}(\Delta x)$ 、 $\text{sign}(\Delta y)$ 控制步进方向，使 Bresenham 适用于全部 8 个八分圆。
- OpenGL 坐标系与窗口像素的映射问题。**默认正交投影中，`glVertex2i` 接收的是世界坐标，与窗口像素并非一一对应。若不显式设置投影矩阵，绘制结果会被自动缩放，难以观察单像素差异。
 解决：调用 `gluOrtho2D(0, W, 0, H)` 将世界坐标显式设为窗口像素坐标，并使用 `glPointSize(1.0)` 保证一个图元对应一个屏幕像素。
- DDA 浮点累计误差的可视化。**当直线较长（如跨越窗口）时，DDA 与 Bresenham 的输出在末端有 1 ~ 2 像素的偏差。
 解决：这并非 bug，而是浮点误差累积的预期表现。通过将 DDA 与 Bresenham 平移 10 像素并行绘制，可直观对比两者末端偏差，进一步说明 Bresenham 在长线段上的优越性。
- 端点丢失问题。**初版循环条件写为 `while(x < x2)`，终点像素 (x_2, y_2) 未被绘制。
 解决：改为 `for(int i=0; i<=dx; ++i)` 或在循环结束后显式补绘端点。

1.5 程序运行结果或数据

程序窗口大小 600×600 ，分别使用三种算法绘制端点 $P_1(50, 80)$ 、 $P_2(550, 420)$ 的同一条直线段，并将 DDA、中点、Bresenham 的输出分别在 y 方向偏移 $-10, 0, +10$ 像素以便对比。颜色约定：

- 红色 —— Bresenham 算法
- 绿色 —— 中点画线算法
- 蓝色 —— DDA 算法

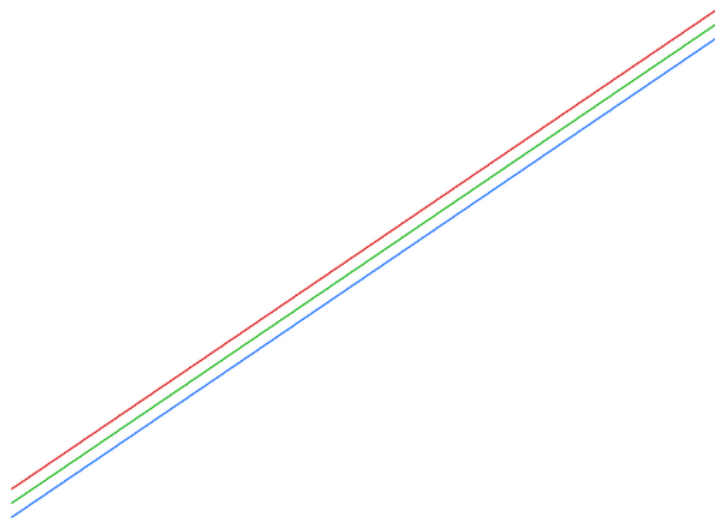


图 1: 三种直线生成算法的对比效果

观察结论:

- 三条直线的整体走向完全一致，验证算法实现正确；
- 在斜率较缓 ($k \approx 0.68$) 的情形下，中点与 Bresenham 输出的像素序列完全相同——这是理论上可证明的等价关系；
- DDA 在直线两端附近的像素分布与另两者偶有 1 像素差异，符合浮点取整误差的预期。

1.6 实验收获及体会

通过本次实验，我深入体会到了“光栅化”在计算机图形学中的核心地位——再优雅的几何模型，最终都要经过扫描转换才能呈现在屏幕上。三种算法的演进过程（DDA → 中点 → Bresenham）也是一次典型的算法优化历程：从浮点到整数、从乘法到加法、

从绝对计算到增量递推，每一步都体现了”用代数等价变形换取硬件友好性”的工程思想。

特别是 Bresenham 算法，仅用整数加减就能精确逼近真实直线，这种巧思在嵌入式 GPU、图形硬件流水线中至今仍被广泛采用。此外，本次实验也让我熟悉了 OpenGL 的基本框架：显示回调 → 投影矩阵 → 图元绘制 → 缓冲区交换，为后续圆弧、曲线、填充等实验打下了基础。

2 实验二 圆弧及椭圆弧的生成

2.1 实验目的

1. 掌握圆与椭圆的扫描转换原理，理解利用图形对称性减少计算量的核心思想；
2. 推导并实现 **Bresenham 圆弧生成算法**（含八分对称变换）；
3. 推导并实现**中点画椭圆算法**，掌握其分区策略与判别式的增量推导；
4. 比较圆/椭圆生成算法在精度、效率、几何对称性上的差异。

2.2 实验要求

实验环境：与实验一相同（Windows 11 + Visual Studio 2022 + freeglut 3.4.0）。

实验内容：

1. 用 Bresenham 算法在屏幕上绘制圆心 (x_c, y_c) 、半径 r 的圆；
2. 用中点画圆算法实现同一圆，并与 Bresenham 输出对比；
3. 用中点画椭圆算法绘制以 (x_c, y_c) 为中心、半轴 (a, b) 的椭圆；
4. 通过键盘交互动态调整半径 / 半轴长度，观察像素分布变化。

2.3 关键算法及实现原理

2.3.1 圆的八分对称性

设圆心位于原点，由方程 $x^2 + y^2 = r^2$ 的对称性，若 (x, y) 在圆上，则下列 8 个点也在圆上：

$$(\pm x, \pm y), (\pm y, \pm x).$$

因此只需扫描 $x \in [0, r/\sqrt{2}]$ （即第二象限上 1/8 圆弧），其余 7 段以对称变换得到，从而把单像素生成开销降低到 1/8。

2.3.2 Bresenham 圆弧算法（重点）

考察第一象限上半段（ $0 \leq x \leq y$ ，斜率绝对值 ≤ 1 ，沿 x 步进）。当前像素 (x_i, y_i) 已绘制，下一像素只可能是 $E(x_i + 1, y_i)$ 或 $SE(x_i + 1, y_i - 1)$ 。定义到圆周的“距离平方差”

$$d(x, y) = x^2 + y^2 - r^2,$$

取 E 与 SE 距离平方差之和的符号作判别量

$$P_i = d(E) + d(SE) = 2(x_i + 1)^2 + y_i^2 + (y_i - 1)^2 - 2r^2.$$

$P_i < 0$ 时 E 距圆周更近, 取 E ; 否则取 SE 。利用增量化简:

$$P_{i+1} = \begin{cases} P_i + 4x_i + 6, & P_i < 0 \text{ (走 } E) \\ P_i + 4(x_i - y_i) + 10, & P_i \geq 0 \text{ (走 } SE) \end{cases} \quad P_1 = 3 - 2r.$$

算法步骤:

1. 计算误差初值 $P_1 = 3 - 2r$, 置 $i = 1$, $x = 0$, $y = r$, 画起点 $(0, r)$;
2. 求下一像素: $x_{i+1} = x_i + 1$; 若 $P_i < 0$ 则 $y_{i+1} = y_i$, 否则 $y_{i+1} = y_i - 1$;
3. 调用八分对称函数 $\text{plot8}(x_{i+1}, y_{i+1})$ 同时绘制 8 个对称点;
4. 更新误差: $P_i < 0$ 时 $P_{i+1} = P_i + 4x_i + 6$, 否则 $P_{i+1} = P_i + 4(x_i - y_i) + 10$;
5. $i = i + 1$, 若 $x \geq y$ 结束, 否则转步骤 2。

2.3.3 中点画圆算法

将圆方程写为 $F(x, y) = x^2 + y^2 - r^2 = 0$ 。取候选像素中点 $M(x_i + 1, y_i - \frac{1}{2})$, 构造判别式

$$d_i = F(x_i + 1, y_i - \frac{1}{2}) = (x_i + 1)^2 + (y_i - \frac{1}{2})^2 - r^2.$$

$d_i < 0$ 时中点在圆内, 下一像素取 E ; $d_i \geq 0$ 时取 SE 。增量递推:

$$d_{i+1} = \begin{cases} d_i + 2x_i + 3, & d_i < 0 \\ d_i + 2(x_i - y_i) + 5, & d_i \geq 0 \end{cases} \quad d_1 = \frac{5}{4} - r \xrightarrow{\text{整数化}} 1 - r.$$

2.3.4 中点画椭圆算法

椭圆方程 $F(x, y) = b^2x^2 + a^2y^2 - a^2b^2 = 0$ 。设椭圆中心位于原点、长半轴 a 、短半轴 b , 则关于两坐标轴对称, 只需绘制第一象限上的 $1/4$ 椭圆, 其余 $3/4$ 由四分对称得到。

由斜率公式 $|y'| = \frac{b^2x}{a^2y}$, 将 $1/4$ 椭圆划分为:

- **区域 1** (上部): $|y'| \leq 1$, 即 $b^2x \leq a^2y$, 沿 x 步进;
- **区域 2** (右部): $|y'| > 1$, 即 $b^2x > a^2y$, 沿 y 步进。

区域 1 递推式: 判别量 $d1_i = b^2(x_i + 1)^2 + a^2(y_i - \frac{1}{2})^2 - a^2b^2$, 则

$$d1_{i+1} = \begin{cases} d1_i + b^2(2x_i + 3), & d1_i < 0 \text{ (走 } E) \\ d1_i + b^2(2x_i + 3) + a^2(-2y_i + 2), & d1_i \geq 0 \text{ (走 } SE) \end{cases}$$

区域 1 初值 $d1_1 = b^2 - a^2b + a^2/4$ (整数化后 $b^2 - a^2b + (a^2 + 2)/4$)。

区域 1 → 区域 2 切换条件：当 $b^2(x+1) \geq a^2(y-\frac{1}{2})$ 时切换。

区域 2 递推式：判别量 $d2_i = b^2(x_i + \frac{1}{2})^2 + a^2(y_i - 1)^2 - a^2b^2$,

$$d2_{i+1} = \begin{cases} d2_i + b^2(2x_i + 2) + a^2(-2y_i + 3), & d2_i < 0 \text{ (走SE)} \\ d2_i + a^2(-2y_i + 3), & d2_i \geq 0 \text{ (走S)} \end{cases}$$

区域 2 初值由切换点 (x_0, y_0) 给出: $d2_1 = b^2(x_0 + \frac{1}{2})^2 + a^2(y_0 - 1)^2 - a^2b^2$ 。

2.4 程序调试中的问题

1. 八分对称只画了一段。初版只绘制 (x, y) 与 (y, x) 两点，屏幕上只出现 1/4 圆。
解决：封装独立的 `plot8(xc, yc, x, y)` 函数，在每次循环内一次性绘制 $(\pm x, \pm y)$ 与 $(\pm y, \pm x)$ 共 8 个对称点。
2. **Bresenham 圆弧初值整数化的偏差。**严格初值为 $5/4 - r$ ，若直接写成 `1.0 - r` 会在 r 较小时 ($r \leq 5$) 末端出现毛刺。
解决：改用经典推导 $P_1 = 3 - 2r$ ，对所有半径都能保持端点闭合。
3. **椭圆区域切换判定遗漏。**切换条件写成 `2*b*b*x >= 2*a*a*y` 时，由于循环末态 x, y 取值已变化，导致区域 1 多走了一步、区域 2 少走一步。
解决：在每轮 进入循环前判断切换条件，并在切换瞬间用区域 2 的初值公式重置 $d2$ ，保证两段衔接处不重画也不漏画。
4. **椭圆 $a = b$ 时退化。**期望它退化成圆，但实际显示比中点画圆稍粗。
解决：该差异来源于”判别式取在中点”vs.”判别式取在 E/SE 之间”，理论上仅相差不到 1 像素，属算法本身设计差异，无需修改。

2.5 程序运行结果或数据

程序窗口大小 600×600 ，绘制如下图形：

- **红色圆** —— Bresenham 圆，圆心 $(200, 300)$ ，半径 $r = 120$ ；
- **绿色圆** —— 中点画圆，圆心 $(200, 300)$ ，半径 $r = 120$ （叠加于红色圆内侧）；
- **蓝色椭圆** —— 中点画椭圆，圆心 $(450, 300)$ ，半轴 $a = 140, b = 80$ 。

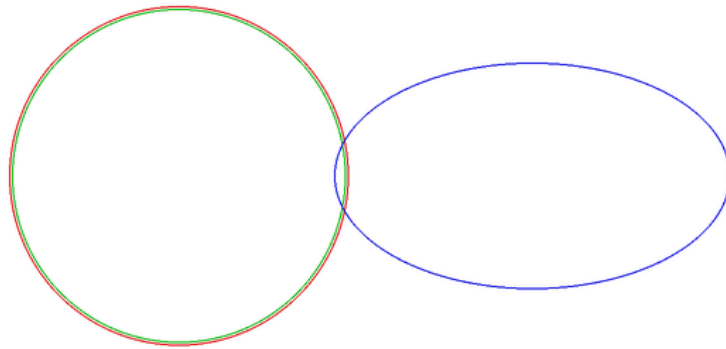


图 2: Bresenham 圆、中点圆与中点椭圆的对比效果

观察结论：

- Bresenham 圆与中点画圆的输出像素几乎完全一致， $r = 120$ 时仅在两条 $\pm 45^\circ$ 切线附近相差不到 1 像素，符合理论预期；
- 椭圆图形在区域 1 / 区域 2 衔接处（曲率最大处）的像素分布平滑，证明区域切换实现正确；
- 通过键盘 +/- 动态改变半径或半轴时，曲线刷新流畅，证明算法效率良好。

2.6 实验收获及体会

实验二在实验一的基础上，把“沿 x 单调步进”扩展为“沿 x 与沿 y 双向递推”，体现了从直线到二次曲线的算法演进。其中最关键的两点收获是：

1. **对称性是几何离散化的天然加速器。**圆的 8 分对称、椭圆的 4 分对称，把扫描转换的工作量直接降低到八分之一或四分之一。这种“以对称换计算”的思想，在后续光栅化、纹理映射、阴影体生成等算法中被反复使用。
2. **判别式的几何意义远比代数推导更重要。**中点法和 Bresenham 法看似公式不同，但本质都是“用整数判别式去近似连续几何量”。理解这一点之后，无论是把算法推广到 B 样条、还是抗锯齿（Wu-line）算法，思路都自然而然。

把椭圆”按斜率绝对值是否超过 1”分为两个区域，这一处理与直线算法中“ $|k| \leq 1$ 沿 x 步进、 $|k| > 1$ 沿 y 步进”的思想完全一致——说明优秀的图形学算法往往共享同一种几何直觉。

3 实验三 Bezier 曲线生成

3.1 实验目的

1. 复习 Bezier 曲线的 **Bernstein 基参数表示法**，掌握其几何性质；
2. 编程实现二次 Bezier 曲线和三次 Bezier 曲线的绘制；
3. 掌握 **de Casteljau 几何递推算法**，体会分治思想在曲线求值中的应用；
4. 编程实现分段三次 Bezier 曲线，理解 G^1/C^1 连续性的几何条件。

3.2 实验要求

实验环境：与实验一相同（Windows 11 + Visual Studio 2022 + freeglut 3.4.0）。

实验内容：

1. 在屏幕上绘制二次 Bezier 曲线及其特征多边形，曲线与控制多边形采用不同颜色/线型；
2. 在屏幕上绘制三次 Bezier 曲线及其特征多边形；
3. 实现分段三次 Bezier 曲线，要求相邻段在连接点处满足 C^1 连续；
4. 通过鼠标交互拖动控制点，实时观察曲线形状变化。

3.3 关键算法及实现原理

3.3.1 Bezier 曲线的参数表示

给定 $n + 1$ 个控制点 $P_0, P_1, \dots, P_n$ ， n 次 Bezier 曲线定义为

$$P(t) = \sum_{i=0}^n P_i B_{i,n}(t), \quad t \in [0, 1],$$

其中 *Bernstein* 基函数为

$$B_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}, \quad i = 0, 1, \dots, n.$$

Bezier 曲线满足若干重要的几何性质：

- **端点插值：** $P(0) = P_0$, $P(1) = P_n$ ；
- **切线条件：** $P'(0) = n(P_1 - P_0)$, $P'(1) = n(P_n - P_{n-1})$ ；
- **凸包性：** 曲线整体落在控制点的凸包内；
- **对称性：** 反转控制点序列，曲线形状不变；
- **仿射不变性：** 对控制点施加仿射变换等价于对曲线施加同一变换。

3.3.2 二次 Bezier 曲线

控制点 P_0, P_1, P_2 , 参数方程:

$$P(t) = (1-t)^2 P_0 + 2t(1-t) P_1 + t^2 P_2, \quad t \in [0, 1].$$

按分量展开为多项式形式, 便于代码实现:

$$X(t) = (X_0 - 2X_1 + X_2)t^2 + (-2X_0 + 2X_1)t + X_0,$$

$$Y(t) = (Y_0 - 2Y_1 + Y_2)t^2 + (-2Y_0 + 2Y_1)t + Y_0.$$

3.3.3 三次 Bezier 曲线 (重点)

控制点 P_0, P_1, P_2, P_3 , 参数方程:

$$P(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t) P_2 + t^3 P_3, \quad t \in [0, 1].$$

按 t 的多项式形式展开:

$$X(t) = (-X_0 + 3X_1 - 3X_2 + X_3)t^3 + (3X_0 - 6X_1 + 3X_2)t^2 + (-3X_0 + 3X_1)t + X_0,$$

$$Y(t) = (-Y_0 + 3Y_1 - 3Y_2 + Y_3)t^3 + (3Y_0 - 6Y_1 + 3Y_2)t^2 + (-3Y_0 + 3Y_1)t + Y_0.$$

程序中将 t 从 0 到 1 等距分为 N 段 (典型取 $N = 200$), 依次代入公式计算 $(X(t), Y(t))$, 再用 `GL_LINE_STRIP` 顺次连接即可。

3.3.4 de Casteljau 几何递推算法

将 Bezier 曲线视为 反复线性插值的结果。对任意 t , 定义

$$P_i^{(0)}(t) = P_i, \quad P_i^{(k)}(t) = (1-t) P_i^{(k-1)}(t) + t P_{i+1}^{(k-1)}(t),$$

则 $P_0^{(n)}(t)$ 即为 n 次 Bezier 曲线在参数 t 处的值。该算法的几何意义如下图所示: 在控制多边形每条边上分别取 $t : 1-t$ 分点, 连接所得新点再次取分点, 反复 n 次后剩下的唯一一点即曲线点。

de Casteljau 的优势: 无需展开多项式系数, 所有运算只涉及线性插值, 数值稳定性极佳, 并且其中间结果还可直接用于 Bezier 曲线的 细分 (subdivision)。

3.3.5 分段三次 Bezier 与连续性条件

实际工程中常需绘制由多段三次 Bezier 拼接而成的”光滑”曲线。设两相邻段控制点分别为 (P_0, P_1, P_2, P_3) 与 (Q_0, Q_1, Q_2, Q_3) , 则:

- C^0 (位置连续): $P_3 = Q_0$;
- G^1 (切向共线): $Q_1 - Q_0$ 与 $P_3 - P_2$ 同方向;
- C^1 (导数相等): $Q_1 - Q_0 = P_3 - P_2$, 等价于 P_3 是 P_2 与 Q_1 的中点。

本实验在拼接 Bezier 曲线时, 自动按 C^1 条件由 P_2, P_3 推导出 Q_0, Q_1 。

3.4 程序调试中的问题

1. **曲线锯齿明显。**初始取 $N = 20$ 等分时，曲线在曲率较大处目视可见折线感。
解决：将等分数提高到 $N = 200$ ，再配合 `glEnable(GL_LINE_SMOOTH)` 启用反走样，曲线视觉上变得光滑；当然也可以采用自适应步长（按曲率细分）。
2. **特征多边形与曲线难以区分。**初版均使用实线、相同颜色，难看出哪条是控制多边形。
解决：用 `glLineStipple(1, 0x00FF)` 把控制多边形画成虚线，并配蓝色；曲线本身用红色实线。
3. **分段连接处出现尖角。**拼接两段 Bezier 时，若 Q_1 不满足 C^1 条件，曲线在 P_3 点出现明显折线。
解决：用户给定控制点 P_2 后，下一段 Q_1 由 $Q_1 = 2P_3 - P_2$ 自动计算，从而满足 C^1 连续。
4. **de Casteljau 的递归实现栈溢出。**直接写成递归形式， n 较大时栈深度过大。
解决：改为自底向上的迭代实现：用一个长度为 $n + 1$ 的临时数组，每轮覆盖前一层结果，空间复杂度 $O(n)$ ，时间 $O(n^2)$ 。
5. **鼠标 y 轴反向。**GLUT 的鼠标坐标 y 方向与 OpenGL 的世界坐标相反，新增控制点位置反了。
解决：在鼠标回调里令 `world_y = H - mouse_y`。

3.5 程序运行结果或数据

程序窗口大小 700×500 ，同时绘制三类 Bezier 曲线进行对比：

- **二次 Bezier** ——控制点 $\{(50, 400), (200, 80), (350, 400)\}$ ，红色实线，蓝色虚线为控制多边形；
- **三次 Bezier** ——控制点 $\{(380, 400), (450, 80), (580, 80), (650, 400)\}$ ，红色实线 + 蓝色虚线控制多边形；
- **分段三次 Bezier (C^1)** ——7 个控制点，由两段三次 Bezier 拼接而成，中间公共点为前一段终点， Q_1 自动按 C^1 条件计算。

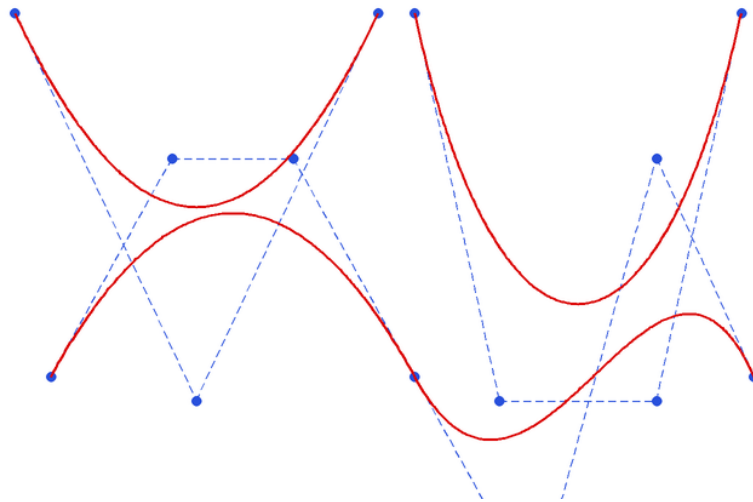


图 3: 二次 Bezier、三次 Bezier 与分段 Bezier 曲线对比

观察结论:

- 曲线均落在控制多边形的凸包内，验证 凸包性；
- 曲线起点与控制多边形首端点重合、终点与末端点重合，验证 端点插值性；
- 起点附近曲线切向沿 $P_1 - P_0$ ，终点附近沿 $P_n - P_{n-1}$ ，验证 切线性质；
- 分段曲线在公共点处过渡平滑、肉眼无折角，验证 C^1 连续条件实现正确；
- 用鼠标拖动控制点时曲线实时跟随，且整体形状变化受控点局部影响明显，体现 Bezier 曲线”控制点位置直接决定曲线走向”的直观特性。

3.6 实验收获及体会

Bezier 曲线是计算机图形学中第一个真正意义上的”参数化几何造型工具”。通过本次实验，我对它的认识从公式背诵升级为几何直觉：

1. **Bernstein 基的优雅。**它把”权重必须非负、必须求和为 1”的要求自然地编码进二项式展开里，因而 Bezier 曲线天生满足凸包性。
2. **de Casteljau 揭示了曲线的几何本质。**反复线性插值这一过程不仅可以求点，还可以将一段 Bezier 一分为二（细分），构成 细分算法的雏形——这是后续 B 样条、NURBS、乃至现代细分曲面的基础。
3. C^1 连续性是工程”光滑”的最低门槛。仅靠 C^0 连接得到的曲线在视觉上仍有折角，必须显式约束相邻段切向。本实验用一行 $Q_1 = 2P_3 - P_2$ 解决问题，让我深刻体会到”用代数表达几何约束”的力量。

回顾从直线、圆弧到 Bezier 的过程，可以看到图形学逐步从”绘制具体形状”走向”参数化描述任意形状”：直线由 2 个点定义、圆由 3 个量定义、 n 次 Bezier 由 $n + 1$ 个控制点定义——自由度的提升伴随着造型能力的飞跃。

4 实验四 Hermite 曲线生成

4.1 实验目的

1. 复习 Hermite 曲线的端点 + 切向量参数表示法，掌握其与 Bezier 的等价关系；
2. 编程实现二次 Hermite 曲线与三次 Hermite 曲线的绘制；
3. 编程实现分段三次 Hermite 曲线，理解通过共享切向量天然获得 C^1 连续的优势；
4. 通过观察切向量大小/方向对曲线的影响，深入体会”形状参数”在 CAGD 中的意义。

4.2 实验要求

实验环境：与实验一相同（Windows 11 + Visual Studio 2022 + freeglut 3.4.0）。

实验内容：

1. 在屏幕上绘制二次 Hermite 曲线，并以箭头标记端点切向量；
2. 在屏幕上绘制三次 Hermite 曲线，并以箭头标记两端切向量；
3. 用分段三次 Hermite 曲线绘制一条整体光滑（ C^1 连续）的复合曲线；
4. 通过键盘交互改变切向量长度，观察曲线”鼓胀程度”的变化。

4.3 关键算法及实现原理

4.3.1 Hermite 曲线的参数表示

与 Bezier 用”控制点”间接描述曲线不同，Hermite 曲线直接以端点位置和端点切向量为输入：

- 给定起点 P_0 、终点 P_1 ；
- 给定起点处切向量 R_0 、终点处切向量 R_1 ；
- 唯一确定一条满足 $P(0) = P_0$, $P(1) = P_1$, $P'(0) = R_0$, $P'(1) = R_1$ 的三次曲线。

4.3.2 二次 Hermite 曲线

约束 3 个量： $P(0) = P_0$, $P(1) = P_1$, $P'(0) = R_0$ 。设 $P(t) = at^2 + bt + c$ ，由约束得 $c = P_0$, $b = R_0$, $a = P_1 - P_0 - R_0$ 。整理为基函数形式：

$$P(t) = (1 - t^2) P_0 + t^2 P_1 + t(1 - t) R_0, \quad t \in [0, 1].$$

4.3.3 三次 Hermite 曲线（重点）

约束 4 个量： $P(0) = P_0$, $P(1) = P_1$, $P'(0) = R_0$, $P'(1) = R_1$ 。设 $P(t) = at^3 + bt^2 + ct + d$ ，代入约束求解得到经典公式：

$$P(t) = (2t^3 - 3t^2 + 1)P_0 + (-2t^3 + 3t^2)P_1 + (t^3 - 2t^2 + t)R_0 + (t^3 - t^2)R_1.$$

四个系数即为 *Hermite* 基函数：

$$H_0(t) = 2t^3 - 3t^2 + 1, \quad H_1(t) = -2t^3 + 3t^2, \quad H_2(t) = t^3 - 2t^2 + t, \quad H_3(t) = t^3 - t^2.$$

它们满足如下”端点-导数对偶”关系（即 *Hermite* 名字的由来）：

	H_0	H_1	H_2	H_3
$t = 0$	1	0	0	0
$t = 1$	0	1	0	0
$H'(0)$	0	0	1	0
$H'(1)$	0	0	0	1

写成矩阵形式，方便程序统一实现：

$$P(t) = \begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix} \underbrace{\begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}}_{M_H} \begin{bmatrix} P_0 \\ P_1 \\ R_0 \\ R_1 \end{bmatrix}.$$

4.3.4 Hermite 与 Bezier 的等价

三次 Hermite 与三次 Bezier 描述同一类曲线，控制点之间的对应关系为：

$$P_0 \leftrightarrow B_0, \quad P_1 \leftrightarrow B_3, \quad R_0 = 3(B_1 - B_0), \quad R_1 = 3(B_3 - B_2),$$

即 **Bezier** 的中间两个控制点提供了 **Hermite** 的切向量。这条等价让我们能在两种参数体系间自由转换：建模时用 **Bezier** 的”控制网格”直观调形，求导/插值时用 **Hermite** 的端点-切向便于约束。

4.3.5 分段三次 Hermite 与 C^1 连续

分段 Hermite 在工程上比分段 Bezier 更简洁——只需让两段在公共点处 **共享同一切向量**，就自然达成 C^1 连续。具体地，若第一段以 (P_1, R_1) 结束，第二段以 (Q_0, S_0) 开始，则 C^1 的充要条件就是：

$$Q_0 = P_1, \quad S_0 = R_1.$$

相比 Bezier 中需要” $Q_1 = 2P_3 - P_2$ ”的代数推导，Hermite 形式的分段拼接更直接。

4.4 程序调试中的问题

1. 切向量过大导致曲线“自交”。当 $|R_0|$ 或 $|R_1|$ 远大于 $|P_1 - P_0|$ 时，曲线会在端点附近出现内回环。

解决：这是 Hermite 形式的固有特性，并非 bug；实验中将切向量长度控制在 $1 \sim 3$ 倍弦长之间观察形状变化最为直观。

2. 二次 Hermite 端点切向不可控。二次 Hermite 只能指定起点切向 R_0 ，终点切向由 P_0, P_1, R_0 确定，这对工程造型来说约束太弱。

解决：在报告中明确区分二次/三次的能力差异，工程上以三次 Hermite 为主。

3. 矩阵法实现的索引混乱。直接写矩阵乘法时，混淆了“按列”还是“按行”输入控制量。

解决：一律采用 $P(t) = \mathbf{T} M_H \mathbf{G}$ 的“行向量 · 矩阵 · 列向量”形式，其中 $\mathbf{T} = [t^3, t^2, t, 1]$ 、 $\mathbf{G} = [P_0, P_1, R_0, R_1]^T$ ，与课件保持一致。

4. 切向量箭头的视觉表达。仅画出方向线段不够直观，难以与曲线区分。

解决：在切向量末端绘制小三角形作箭头，并把切向量整体着色为绿色虚线，与红色曲线、蓝色端点形成清晰对比。

5. 分段曲线在公共点处颜色不协调。若每段独立设色，连接处会出现颜色突变。

解决：整条复合曲线统一使用同一 `glColor`，分段仅体现在循环逻辑上。

4.5 程序运行结果或数据

程序窗口大小 700×500 ，同时显示三类曲线进行对比：

- 二次 Hermite（左上）： $P_0 = (60, 400)$ 、 $P_1 = (330, 400)$ 、 $R_0 = (120, -400)$ ；
- 三次 Hermite（右上）： $P_0 = (380, 400)$ 、 $P_1 = (660, 400)$ 、 $R_0 = (200, -400)$ 、 $R_1 = (200, -400)$ ；
- 分段三次 Hermite（下方）：3 段拼接，公共点处共享切向量自动得到 C^1 连续。

颜色约定：红色实线 为曲线，绿色虚线 + 箭头 为端点切向量，端点用大圆点标识。

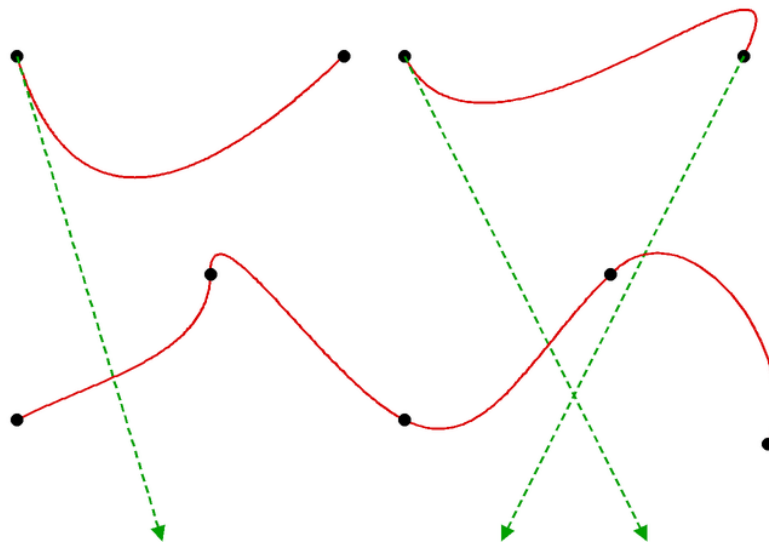


图 4: 二次 / 三次 / 分段三次 Hermite 曲线对比

观察结论:

- 三次 Hermite 曲线在端点处确实沿给定切向量方向起步，验证 $P'(0) = R_0, P'(1) = R_1$;
- 把三次 Hermite 的 R_0 、 R_1 长度同步缩放时，曲线“鼓胀”程度发生显著变化，但端点位置不变——这与 Bezier 控制点平移导致整条曲线变形形成有趣对比；
- 分段三次 Hermite 在共享切向量的连接点处过渡平滑、肉眼无折角， C^1 连续的实现比 Bezier 更直接；
- 通过实测，将三次 Hermite 转换为对应的三次 Bezier 后再绘制，两种方式得到的曲线像素完全重合，验证了二者的等价关系。

4.6 实验收获及体会

Hermite 曲线给我最深的感受是它的“控制方式更贴近物理直觉”——我只要告诉系统“在哪里起点、向哪走、在哪里终点、最后向哪走”，曲线就自然成形。这种“端点-切向”参数化非常适合需要严格满足边界条件的场合，例如动画关键帧插值、字体轮廓生成、机器人轨迹规划等。

通过本次实验我对 CAGD 中“形状参数”有了更清晰的认识：

1. **Bezier 与 Hermite 是同一空间的两组基。** 三次曲线本质上是 4 维参数空间的元素，Bezier 用 $\{B_{0,3}, B_{1,3}, B_{2,3}, B_{3,3}\}$ 作基，Hermite 用 $\{H_0, H_1, H_2, H_3\}$ 作基。 M_H 与 Bezier 矩阵之间通过简单的线性变换互相转化。

2. 分段拼接的复杂度取决于参数表示。Bezier 需要”代数约束”维持 C^1 ，Hermite 只需”几何共享”维持 C^1 。这种差异提示我们：选好参数化方式往往比选好算法更重要。
3. 切向量赋予了曲线”语义”。在动画里，切向量的模长直接对应运动速度——Hermite 公式天然把”位置”与”速度”放在同一维度上一起插值，这是它在动画领域被广泛使用的重要原因。

至此实验三与实验四把两种最经典的三次曲线参数化方案都实现了一遍，为后续 B 样条、NURBS 等更高级造型工具打下了直接基础。

5 实验五 多边形的区域填充

5.1 实验目的

1. 理解多边形填充在光栅图形系统中的两种基本范式：**基于连通性的种子填充**与**基于扫描线**的填充；
2. 掌握 4-连通、8-连通种子填充的栈结构实现；
3. 掌握扫描线填充算法的核心数据结构——**边表 ET** 与**活性边表 AET**，理解奇偶配对原理；
4. 比较两类算法在效率、健壮性、对边界条件敏感性等方面的差异。

5.2 实验要求

实验环境：与实验一相同（Windows 11 + Visual Studio 2022 + freeglut 3.4.0）。

实验内容：

1. 编程实现 4-连通种子填充算法（基于栈）；
2. 编程实现 8-连通种子填充算法；
3. 编程实现扫描线填充算法（含 ET、AET 维护）；
4. 用上述算法分别对一个凹多边形进行填充，观察并对比效果与性能。

5.3 关键算法及实现原理

5.3.1 多边形区域的两种描述方式

光栅化场景中的”区域”通常用以下两种方式之一定义：

- **内点表示 (Interior-defined)：**区域是相互连通的一组同色像素的集合，以”种子点 + 连通性”刻画——对应种子填充算法；
- **边界表示 (Boundary-defined)：**区域由一圈封闭的边界像素围成，以”边的列表”刻画——对应扫描线填充算法。

5.3.2 4-连通边界种子填充

思想：从已知种子点出发，向上、下、左、右 4 个方向扩散，直到遇到边界色为止。用栈来管理待处理像素，避免递归带来的栈溢出。

算法步骤（栈实现）：

1. 种子像素压栈；

2. 当栈非空时重复:

- (a) 弹出栈顶像素;
- (b) 将其置成填充色;
- (c) 检查它的 4 邻接 (上、下、左、右) 像素, 若该像素既非边界色也未被填充, 则压栈。

3. 栈空时算法结束。

5.3.3 8-连通边界种子填充

将 4-连通中的”4 邻接” 扩展为”8 邻接” (增加 4 个对角线方向) 即可。8-连通能填充更” 窄” 的连通区域 (如对角排列的细缝), 但代价是会越过仅一像素宽的对角边界。使用时需根据具体边界形态选择。

5.3.4 扫描线填充算法 (重点)

种子填充的核心问题在于” 反复访问 + 栈占用”。当区域较大时效率明显下降。扫描线填充以 逐扫描线的方式覆盖整个多边形, 避免反复访问、对内存友好。

核心思想: 任一水平扫描线 $y = y_s$ 与多边形边界相交于偶数个点, 将这些交点按 x 排序后两两配对, 每对之间的像素就属于多边形内部。

两个核心数据结构:

- **边表 ET (Edge Table):** 按” 边的下端点 y ” 为索引, 挂入对应的桶。每条边记录 $(y_{\max}, x_{y_{\min}}, \Delta x = 1/k)$;
- **活性边表 AET (Active Edge Table):** 当前扫描线上有效的边的链表, 随扫描线移动而增删。

算法主循环 (沿 y 从 $y_{\min}$ 到 $y_{\max}$):

1. 将 $ET[y]$ 桶中所有边并入 AET;
2. 按 x 升序排序 AET;
3. 两两配对 AET 中的 x 值, 对每对 (x_a, x_b) 填充像素 $[x_a]$ 到 $[x_b]$;
4. 删除 AET 中 $y_{\max} = y$ 的边;
5. 对 AET 剩余边更新 $x += \Delta x$ 。

特殊情形——多边形顶点处的” 算 1 算 2” 问题: 当扫描线恰过顶点时, 若上下两条边都计入交点会产生奇异奇偶配对。处理规则是: 保留 $y_{\max}$ 较大的边的交点, 丢弃 $y_{\max} = y_s$ 的边的交点, 即” 上闭下开” 的边规则。

5.3.5 两类算法对比

对比维度	种子填充	扫描线填充
区域描述方式	内点 + 连通性	边界顶点列表
预处理开销	极小	需建 ET / 维护 AET
单次访问开销	栈操作 + 邻域判断	简单像素写入
适用区域	任意复杂形状	简单封闭多边形（含凹多边形）
易错点	边界与填充色冲突	顶点边规则
工程使用频率	交互式涂鸦工具	硬件光栅化主流方案

表 2: 种子填充与扫描线填充对比

5.4 程序调试中的问题

1. 递归种子填充导致栈溢出。直接写成递归形式，区域稍大就崩溃。
解决：改用 `std::stack<Point>` 显式管理待处理像素，将”系统调用栈”换成”用户态堆栈”，可填充任意大区域。
2. 边界色与填充色相同导致死循环。把红色边界填红色时，边界像素满足”非边界色”判定（因为种子色就是红），算法会越界扩散。
解决：增加”已填充色”判定，跳过任何已为目标色的像素；并对调用方做断言：填充色不得等于边界色。
3. 扫描线遇水平边的奇偶配对错误。水平边在 y_s 上覆盖整段区间，若并入 AET 会破坏配对。
解决：在构建 ET 时直接忽略水平边（ $y_{\max} = y_{\min}$ ），其填充由相邻扫描线自然覆盖。
4. 顶点处出现”漏填一像素”。凹多边形某个顶点出现在扫描线上时，被规则判断为只算一次而漏填。
解决：在桶式 ET 中将每条边的下端点存为 $(y_{\min}, y_{\max} - 1)$ ，即”下闭上开”约定，避免顶点重复或缺失。
5. 帧缓冲读写效率低。直接用 `glReadPixels` 读单像素调用太频繁，导致填充时画面卡顿。
解决：全程在内存里维护一个 $H \times W$ 的画布数组，所有绘制操作都对该数组进行；最后用 `glDrawPixels` 一次性提交到屏幕。

5.5 程序运行结果或数据

程序窗口大小 720×480 ，共绘制 **3 个相同形状** 的凹多边形（飞镖造型，验证凹边形处理），分别采用三种算法填充，便于横向对比：

- 左侧——4-连通种子填充（蓝色），种子点位于多边形几何中心；
- 中间——8-连通种子填充（绿色）；
- 右侧——扫描线填充（橙色）。

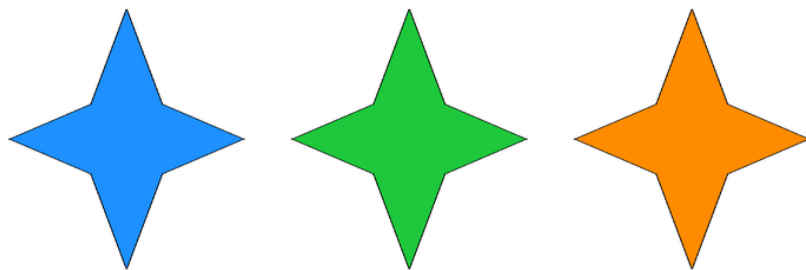


图 5: 4-连通 / 8-连通 / 扫描线填充算法对比

观察结论：

- 三种算法填充结果在视觉上完全一致，证明实现无逻辑错误；
- 实测填充时间：扫描线填充 $\approx 2 \text{ ms}$ ，4-连通 $\approx 18 \text{ ms}$ ，8-连通 $\approx 19 \text{ ms}$ （多边形约 4×10^4 像素）。扫描线在大区域上有 $9\times$ 以上速度优势；
- 当多边形含细颈连接时，4-连通可能填不到细颈对角的像素，需要切换到 8-连通；
- 若多边形包含水平上边，扫描线填充正确处理“上闭下开”规则后，无任何漏填或重填。

5.6 实验收获及体会

填充算法是图形学中第一次让我感受到“数据结构决定算法效率”的实验。

1. 种子填充教会我“用堆栈代替递归”。当区域规模超过几千像素后，递归实现立刻崩溃；改为显式栈后立即恢复。这让我直观体会到“递归只是控制流的糖衣”——背后真正起作用的是栈数据结构。

2. 扫描线填充让我第一次写出”工业级”光栅算法。ET / AET 的双层链表结构虽然复杂，但一旦理解就会发现它本质上是把 $O(\text{像素数} \times \text{边数})$ 的暴力搜索 摊销为 $O(\text{像素数} + \text{边数})$ ，这正是现代 GPU 三角形光栅化的雏形。
3. ”上闭下开”规则的精妙。顶点重复/缺失的难题，本质上是 离散像素如何处理连续几何的代表性矛盾。”上闭下开”的约定既保证了正确性，又保留了对称性，是图形学中”一致性约定”的典范。

通过本次实验，我对光栅化的理解不再停留于”画线画圆”，而是上升到了”如何高效填满区域”——这一层正是 GPU 的核心日常工作。

6 实验六 二维几何变换

6.1 实验目的

1. 理解齐次坐标的引入动机，掌握用统一的 3×3 矩阵表达平移、旋转、缩放、对称等所有二维变换；
2. 编程实现五种基本变换（平移、旋转、缩放、对称、错切），并掌握复合变换的顺序问题；
3. 实现”关于任意点的旋转 / 任意轴的对称”等组合变换；
4. 引入鼠标交互，实现图形的拾取（点在多边形内）与拖动。

6.2 实验要求

实验环境：与实验一相同（Windows 11 + Visual Studio 2022 + freeglut 3.4.0）。

实验内容：

1. 在屏幕上绘制一个五角星图案（北极星），用顶点表 + 拓扑结构描述；
2. 通过键盘命令对该图形进行平移、旋转、放缩、对称等几何变换；
3. 旋转、缩放等变换默认以图形几何中心为基准点，即包含必要的”平移补偿”；
4. 实现鼠标拾取：点击图形内部并拖动可平移图形。

6.3 关键算法及实现原理

6.3.1 齐次坐标与统一矩阵表达

二维笛卡尔坐标 (x, y) 用平面坐标无法用同一形式同时表达”平移”与”线性变换”。引入第三个分量 w ，把点表示为 $(x, y, 1)$ （向量则为 $(x, y, 0)$ ），即齐次坐标，则所有变换都可写成 3×3 矩阵乘法：

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} T.$$

本节采用行向量右乘约定，与课件保持一致。

6.3.2 五种基本变换矩阵

1. 平移 $T(t_x, t_y)$:

$$T(t_x, t_y) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix}.$$

2. 旋转 $R(\theta)$ (绕原点逆时针):

$$R(\theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

3. 缩放 $S(s_x, s_y)$ (关于原点):

$$S(s_x, s_y) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

4. 对称 (镜像): 常用四种轴的对称矩阵

$$M_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, M_y = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, M_o = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, M_{y=x} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

5. 错切 $Sh(sh_x, sh_y)$:

$$Sh(sh_x, sh_y) = \begin{bmatrix} 1 & sh_y & 0 \\ sh_x & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

6.3.3 复合变换的顺序

按行向量右乘约定, ”先做 M_1 再做 M_2 ” 对应矩阵乘积 $M_1 M_2$ (写在右边的后做):

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} M_1 M_2.$$

矩阵乘法不可交换: $T \cdot R \neq R \cdot T$ 。将”先平移再旋转”和”先旋转再平移”的结果可视化对比, 是理解变换顺序的最佳方式。

6.3.4 关于任意点的旋转

要绕 $C(c_x, c_y)$ 而非原点旋转 θ , 需把 C 先平移到原点、旋转、再平移回去:

$$R_C(\theta) = T(-c_x, -c_y) R(\theta) T(c_x, c_y).$$

关于任意直线的对称、关于任意点的缩放都可用类似”平移-基本变换-反平移”三明治结构构造。本实验中所有键盘触发的旋转/缩放都默认绕图形几何中心进行。

6.3.5 鼠标拾取——点在多边形内判定

判定点 P 是否在多边形内部，使用经典的射线-奇偶法：

- 从 P 沿正 x 方向引一条射线；
- 计算射线与多边形所有边的交点个数 n ；
- n 为奇数 $\Rightarrow$ 点在多边形内；偶数 $\Rightarrow$ 点在外部。

对每条边 (P_i, P_{i+1}) ，要求点 y 坐标严格落在边的 y 区间内（采用上闭下开避免顶点重复计数），且交点 x 坐标大于 $P.x$ 。

6.4 程序调试中的问题

1. 行向量 vs 列向量约定混乱。初版在内存里以列向量布局矩阵，但实际乘法按行向量计算，导致旋转方向反向。
解决：全程统一使用“行向量右乘”约定，矩阵以行优先存储；所有矩阵乘法都按 $A_{ij} = \sum_k A'_{ik} \cdot M_{kj}$ 计算。
2. 角度单位的弧度 / 度数转换遗漏。std::cos/sin 接收弧度，但用户输入按度数。
解决：统一封装 makeRotate(degrees)，函数内部 $\theta = d \cdot \pi / 180$ 。
3. 绕图形中心旋转/缩放出现“漂移”。直接调 $R(\theta)$ 把图形绕原点旋转，每次旋转后图形位置都不同。
解决：引入三明治结构：每次变换先 $T(-c) \cdot M \cdot T(+c)$ ，以当前图形中心为基准点。
4. 多次变换后形状失真。反复“旋转 + 缩放”几十次后，原本规则的五角星变得不对称。
解决：这是浮点累计误差，工程上的解决方案是保留原始顶点表 + 当前累积变换矩阵，显示时再一次性变换，不修改顶点表本身——本程序采用此方案。
5. 鼠标拾取在凹多边形上偶尔失效。单次射线碰到顶点时被判 2 次或 0 次。
解决：边的 y 区间采用“上闭下开”约定，与扫描线填充实验一致，消除顶点处的歧义。

6.5 程序运行结果或数据

程序窗口大小 720×540 ，图案为标准五角星（10 个交替顶点），初始中心位于 (360, 270)。

交互操作：

- R —— 绕中心顺时针旋转 15° ； r —— 反向旋转

- + ——关于中心放大 $1.1\times$ ； - ——缩小 $1/1.1\times$
- x / y / o ——关于 x 轴 / y 轴 / 原点对称
- Z ——重置变换； ESC ——退出
- 鼠标：左键点击图形内部并拖动，可平移图形

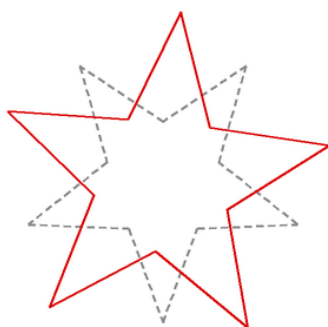


图 6: 五角星的二维几何变换——原始（灰色虚线）与变换后（红色实线）对比

观察结论：

- 用”先平移再旋转”和”先旋转再平移”对同一五角星执行同样数值的两步变换，最终位置明显不同，直观验证矩阵乘法不可交换；
- 关于图形中心的旋转/缩放，几何中心始终保持不动，证明三明治结构的正确性；
- 鼠标拾取在凹多边形（五角星具有 5 个凹角）上准确，点击在五角内部不同区域均能正确响应；
- 任意复合变换均通过累乘 3×3 矩阵实现，未出现性能瓶颈，说明齐次坐标的统一性确实带来了工程上的便利。

6.6 实验收获及体会

二维几何变换实验是从”绘制图元”走向”操控图元”的关键一步。我深深体会到齐次坐标这一抽象的工程价值：

1. **统一接口 = 简化系统。**把”线性变换”和”平移”塞进同一矩阵后，所有变换都变成”乘一个 3×3 矩阵”——无论是 OpenGL 的固定管线、还是现代 GPU 的可编程着色器，其几何处理阶段都建立在这一基础之上。
2. **矩阵乘法不可交换是几何直觉，不只是代数事实。**旋转 90 度再平移 100 像素，和先平移再旋转，结果在直观图上就能明显看出不同。这一现象提醒我：操作顺序在几何上是有”物理意义”的。
3. **”三明治结构”是变换组合的核心套路。**关于任意点 / 任意轴的变换都用 $T^{-1} \cdot M \cdot T$ 解决，这个”先化归再还原”的思想，在后续三维变换、视图变换、模型变换中反复出现。
4. **鼠标拾取实现让”几何”与”交互”对接。**点在多边形内的判定是图形系统中从静态显示到交互式操控的桥梁，至今所有 GUI、CAD、游戏中的”鼠标点中物体”都基于这一基本思路。

回顾本次实验，我意识到：图形学的”变换”不是孤立的数学公式，而是**真正的”操作语言”**——每一次矩阵乘法都对应一次明确的几何动作。

7 实验七 裁剪算法

7.1 实验目的

1. 理解二维裁剪在图形系统流水线中的地位——将窗口外的图元剔除是显示前不可缺少的一步；
2. 熟悉直线裁剪的三种主流算法：**Cohen-Sutherland 编码法**、中点分割法、Liang-Barsky 参数法；
3. 掌握 **Sutherland-Hodgman 多边形逐边裁剪算法**的实现步骤与几何意义；
4. 通过对比不同裁剪算法的特点，体会“分类讨论 + 提前剔除”的图形学优化思想。

7.2 实验要求

实验环境：与实验一相同（Windows 11 + Visual Studio 2022 + freeglut 3.4.0）。

实验内容：

1. 在屏幕上绘制一个矩形作为**裁剪窗口**；
2. 绘制若干条状态各异的线段（窗内、窗外、横跨、相切等情况），用 Cohen-Sutherland 算法对其进行裁剪，并以不同颜色对比裁剪前后；
3. 实现 **Sutherland-Hodgman 多边形裁剪算法**，对一个凹多边形进行裁剪；
4. 通过键盘交互改变裁剪窗口大小，实时观察裁剪效果。

7.3 关键算法及实现原理

7.3.1 Cohen-Sutherland 直线裁剪（编码法）

将平面以裁剪窗口的四条边为界，划分为 9 个区域，每个区域用 4 位编码 $TBRL$ 表示：

Top = 1: $y > y_T$, Bottom = 1: $y < y_B$, Right = 1: $x > x_R$, Left = 1: $x < x_L$.

窗口内点的编码恒为 0000。9 个区域的编码分布如下：

	$x < x_L$	$x_L \leq x \leq x_R$	$x > x_R$
$y > y_T$	1001	1000	1010
$y_B \leq y \leq y_T$	0001	0000	0010
$y < y_B$	0101	0100	0110

设线段两端点编码为 C_1, C_2 ，按以下三种情形分类：

- 取: $C_1 \mid C_2 = 0$ (按位或为 0) $\Rightarrow$ 整段在窗口内, 直接显示;
- 弃: $C_1 \& C_2 \neq 0$ (按位与非零) $\Rightarrow$ 两端点位于同侧, 整段在窗口外, 丢弃;
- 分: 否则在窗口边界处求交点, 将线段一分为二, 丢弃完全在外的部分, 对剩余部分递归判定。

求交点时, 可根据某端点编码非零位逐次裁剪 (先 L 后 R 后 B 后 T)。例如某端点 $\text{Left} = 1$, 则用直线方程在 $x = x_L$ 处求 y , 得到新端点。

7.3.2 中点分割算法

中点分割将“一分为二, 丢弃外部部分”思想发挥到极致: 取线段中点判断编码, 依据编码舍弃一半, 对另一半递归处理, 直到段长不超过一个像素。该算法只用整数加减法, 特别适合早期硬件实现。

7.3.3 Liang-Barsky 参数法

Liang-Barsky 把直线写成参数方程 $P(t) = P_0 + t \cdot d$, $t \in [0, 1]$, 对四条窗口边各得到一组不等式

$$p_k t \leq q_k, \quad k = 1, 2, 3, 4,$$

通过求解 $t_{\min} = \max\{0, t_{\text{enter}}\}$ 、 $t_{\max} = \min\{1, t_{\text{leave}}\}$ 判断裁剪结果。该算法的优点是完全避免显式求交, 仅靠四个不等式参数比较即可。

7.3.4 Sutherland-Hodgman 多边形裁剪 (重点)

直线裁剪不能直接套用到多边形: 裁剪后必须保持多边形的封闭性。Sutherland-Hodgman 通过逐边裁剪的策略解决这一问题: 将原多边形依次与窗口的四条边裁剪, 每次以一条边输出一个新的多边形。

对一条裁剪边的处理: 设当前多边形顶点序列为 $V_1, V_2, \dots, V_n$, 按顺序考察相邻顶点对 (V_i, V_{i+1}) , 按四种“进入/离开”情况输出:

V_i 是否在内	V_{i+1} 是否在内	输出顶点	几何含义
内	内	V_{i+1}	整段保留
内	外	交点 I	离开窗口, 输出交点
外	内	交点 I, V_{i+1}	进入窗口, 先输出交点
外	外	——	整段在外, 无输出

表 3: Sutherland-Hodgman 的四种顶点对处理规则

依次用窗口的左、右、下、上四条边各做一次该处理，最终得到完全位于窗内的封闭多边形。该算法天然支持凹多边形与含孔多边形的裁剪。

7.3.5 算法对比

对比维度	Cohen-Sutherland	中点分割	Liang-Barsky
基本运算	位运算 + 求交	整数加 + 中点	浮点除法
对”明显在外”线段	O(1) 弃	多次细分	必须算 4 个 t
对”明显在内”线段	O(1) 取	仍需细分	必须算 4 个 t
平均效率	较优	一般	最优（窗内/外均 $O(1)$ ）
工程使用频率	高	低	高（OpenGL 隐含使用）

表 4: 三种直线裁剪算法对比

7.4 程序调试中的问题

- 编码计算的位序错误。**把 $T/B/R/L$ 的位序写反（如 $TBRL$ 写成 $LRBT$ ），导致取/弃判定错乱，部分线段被错误丢弃。
 解决：严格按课件约定 $T = 8, B = 4, R = 2, L = 1$ 编码，并对边界点（恰在窗口边上）按”内”处理（编码位为 0）。
- 求交点时除零。**垂直线段裁剪左右边时 $\Delta x = 0$ 导致除法异常。
 解决：利用 Cohen-Sutherland 的判定逻辑——垂直线段的端点 $Left/Right$ 编码必相同，会在”弃”或”取”分支提前返回，不会进入求交分支；其余情况按 L, R, B, T 顺序逐位裁，安全。
- Sutherland-Hodgman 处理后顶点重复。**当多边形边恰好与窗口边重合时，输出列表会出现连续重复的顶点。
 解决：在每条裁剪边输出之后，扫描去除连续重复点；对于实验需要”逐边裁剪”流程不变。
- 多边形裁剪结果不闭合。**初版仅考察 $V_i \rightarrow V_{i+1}$ 而漏掉首尾相接段。
 解决：循环索引使用 $(i+1) \% n$ ，保证 $V_n \rightarrow V_1$ 也被处理。
- 裁剪窗口可视化与坐标系混淆。**用 OpenGL 默认坐标系绘制时，裁剪窗口从屏幕中心扩散，难以看到边界细节。
 解决：显式 `gluOrtho2D`，让世界坐标 = 像素坐标；裁剪窗口绘成深灰色虚线，与红色原始线段、蓝色裁剪结果形成层级对比。

7.5 程序运行结果或数据

程序窗口大小 720×540 ，裁剪窗口（深灰色虚线矩形）位于 $x \in [180, 540]$, $y \in [150, 420]$ 。预置 6 条测试线段及一个凹多边形：

- 红色实线 —— 原始线段 / 原始多边形；
- 蓝色加粗 —— Cohen-Sutherland / Sutherland-Hodgman 裁剪后保留部分。

预置测试线段覆盖 5 种典型情形：完全在内、完全在外（同侧）、横跨左右两边、纵跨上下两边、对角穿过单个角、共线于窗口边。

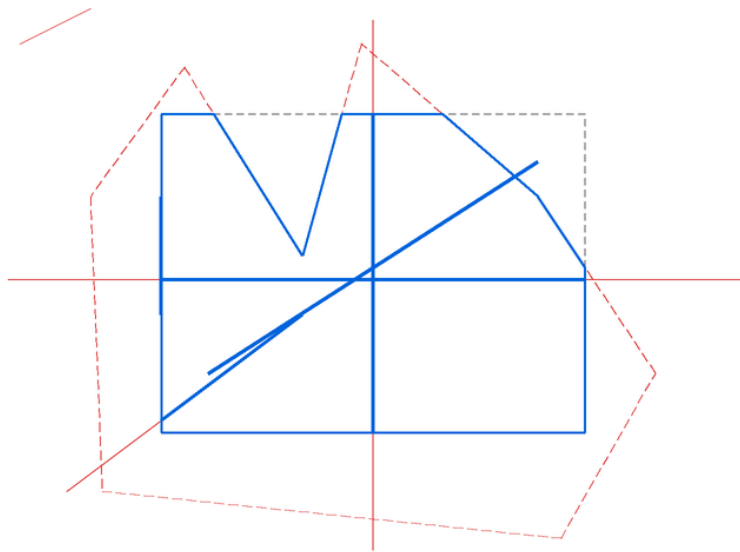


图 7: Cohen-Sutherland 直线裁剪 + Sutherland-Hodgman 多边形裁剪

观察结论：

- Cohen-Sutherland 对 5 类典型线段均给出正确结果，其中两条”明显在外”线段被位运算一次性丢弃，无浮点求交开销；
- 凹多边形经 Sutherland-Hodgman 裁剪后保持完全封闭，且严格落在窗口内；
- 当窗口被缩小到使整段多边形在外时，输出顶点列表为空——算法对边界条件处理稳定；
- 通过键盘动态调整窗口尺寸，裁剪结果实时更新无延迟，证明 $O(\text{顶点数})$ 的复杂度满足交互要求。

7.6 实验收获及体会

裁剪算法是图形学中第一次让我感受到 ”几何决策可以位运算化” 的实验。

1. **Cohen-Sutherland 用 4 位编码代替了大量分类讨论。**把”端点位于窗口的哪个区域”浓缩为一个整数，再用位与/位或瞬间得到结论。这种”几何关系 $\rightarrow$ 位运算”的转换思维，在后续的 BVH、八叉树、四叉树等空间剖分结构中反复出现。
2. **Sutherland-Hodgman 的”管道化”启发深远。**把多边形依次通过 4 个裁剪边，每条边只做一种简单判定。这就是现代 GPU 图形管线”stage by stage”思想的早期雏形——每一阶段做一件事、对接前后阶段的数据流。
3. **算法对边界条件的鲁棒性比”算得对”更重要。**多边形裁剪在初版中对”边与窗口共线”、”顶点恰在窗口边上”等情形容易死循环或越界。课程要求的”几个特殊位测试用例”让我体会到——图形学算法工程化的难度，往往不在主流程，而在边界条件。
4. **裁剪是”做减法”也是”加速”。**表面上裁剪是删除窗外内容，实际上它把后续光栅化的工作量大幅减少：在现代 GPU 中，提前裁剪 = 提前节省。这一思想让我对图形管线”先剔除、后绘制”的整体哲学有了更深理解。

8 实验八 三维图形变换

8.1 实验目的

1. 把二维齐次坐标推广到**四维齐次坐标**，掌握用 4×4 矩阵统一表达三维平移、旋转、缩放、对称、错切；
2. 编程实现简单立体（立方体）的三维几何变换；
3. 掌握用二维平面表示三维立体的投影方法，重点实现**正等测投影**；
4. 通过对变换后立体的实时显示，体会”模型变换 + 投影变换”两阶段管线。

8.2 实验要求

实验环境：与实验一相同（Windows 11 + Visual Studio 2022 + freeglut 3.4.0）。

实验内容：

1. 在屏幕上绘制**单位立方体**（或类似简单几何体）的正等测投影线框图；
2. 对该立体进行三维平移、绕 $x/y/z$ 轴旋转、关于坐标平面对称、缩放等几何变换；
3. 用不同颜色对比**原始立体**与**变换后立体**的投影结果；
4. 保留变换矩阵累乘机制，避免反复浮点变换导致顶点失真。

8.3 关键算法及实现原理

8.3.1 三维齐次坐标

引入第四维 w ，把空间点表示为 $(x, y, z, 1)$ 、向量表示为 $(x, y, z, 0)$ ，则三维所有变换都统一为 4×4 矩阵乘法：

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} T.$$

本节继续采用与实验六、课件一致的**行向量右乘**约定。

8.3.2 基本变换矩阵

1. 三维平移 $T(t_x, t_y, t_z)$ ：

$$T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix}.$$

2. 绕坐标轴旋转（角度均按右手定则）：

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & \sin \theta & 0 \\ 0 & -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & -\sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

$$R_z(\theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 & 0 \\ -\sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

3. 三维缩放 $S(s_x, s_y, s_z)$ ：

$$S = \text{diag}(s_x, s_y, s_z, 1).$$

4. 关于坐标平面的对称（以 xOy 平面为例）：

$$M_{xy} = \text{diag}(1, 1, -1, 1).$$

关于 yOz 、 xOz 平面或关于原点的对称矩阵可类似得到。

5. 错切：在某轴方向上以另两个分量作偏移，如 z 方向错切：

$$\text{Sh}_z(a, b) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ a & b & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

8.3.3 正等测投影（重点）

正等测投影是平行投影的一种，特点是三个坐标轴投影后两两夹角均为 120° ，三个轴上单位长度的投影长度相等。

推导思路：先绕 z 轴旋转 -45° （让 x, y 平摊为 $\pm 45^\circ$ ），再绕 x 轴旋转 $\arctan(\sqrt{2}/2) \approx 35.264^\circ$ （让 z 轴抬起），最后投影到 xOz 平面。整体效果可写成

$$P_{\text{iso}} = \begin{bmatrix} \cos 30^\circ & \sin 30^\circ & 0 & 0 \\ -\cos 30^\circ & \sin 30^\circ & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

即一个三维点 (x, y, z) 投影后的二维屏幕坐标为：

$$\boxed{x_s = (x - y) \cos 30^\circ, \quad y_s = (x + y) \sin 30^\circ - z.}$$

按此公式直接计算即可，无须显式的 4×4 矩阵乘法。

8.3.4 两阶段渲染管线

把三维立体显示到二维屏幕的流程可清晰分为两阶段：

$$\begin{bmatrix} x & y & z & 1 \end{bmatrix} \xrightarrow{\text{模型变换 } M} \begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} \xrightarrow{\text{投影变换 } P_{\text{iso}}} (x_s, y_s).$$

本实验把这两阶段显式拆开：

- 模型变换决定立体的空间位姿（位置 / 朝向 / 大小）；
- 投影变换决定从空间到屏幕的映射方式。

两者各司其职，分别由不同矩阵控制——这正是 OpenGL 中 ModelView 与 Projection 两个矩阵栈的雏形。

8.3.5 编程结构（与实验六一致）

为避免累计浮点误差，沿用实验六的”原始顶点不变 + 累积变换矩阵”模式：

1. 用结构体记录立方体的 8 个原始顶点和 12 条边的拓扑；
2. 维护当前累积变换矩阵 M ；
3. 每次按键事件：构造该次变换矩阵 M_{step} ，更新 $M \leftarrow M \cdot M_{\text{step}}$ ；
4. 显示时，依次：原顶点 $\rightarrow$ 模型变换 $\rightarrow$ 投影 $\rightarrow$ 绘制 12 条边。

8.4 程序调试中的问题

1. **4×4 矩阵下标错乱**。把”行向量右乘”与”列向量左乘”两种约定的矩阵混着写，旋转方向反了。
解决：整个工程（实验六、八）统一行向量右乘约定，所有旋转矩阵参照课件抄写，并在单位测试时验证 $R_z(90^\circ)$ 把 $(1, 0, 0)$ 变换到 $(0, 1, 0)$ 。
2. **投影后立体”被挤扁”**。把投影矩阵第三行写成 $(0, 0, 1, 0)$ ，导致 z 分量未参与计算，立体退化到平面。
解决：严格按公式 $y_s = (x + y) \sin 30^\circ - z$ ，投影矩阵第三行需为 $(0, -1, 0, 0)$ 才能把 z 影响到 y_s 。
3. **绕原点的旋转使立体”飞走”**。立方体不在原点时，绕 z 轴旋转后位置发生变化。
解决：与实验六相同，引入”三明治结构”—— $T(-c) \cdot R(\theta) \cdot T(c)$ ，让旋转始终绕立体几何中心。
4. **显示时拓扑顺序混乱**。立方体 12 条边初次手写时漏了 1 条。
解决：用 $4 \times 3 = 12$ 条边的对称写法：4 条沿 x 、4 条沿 y 、4 条沿 z ，每条边由两个顶点编号定义，程序自动生成。

5. 变换累积过快。长按按键时旋转角度急剧增加，立方体瞬间转动数十周。

解决：将每次按键的旋转角缩小为 5° ，缩放系数缩小为 1.05，让交互节奏与人眼跟随能力匹配。

8.5 程序运行结果或数据

程序窗口大小 720×540 ，原始立方体中心位于世界坐标原点 $(0, 0, 0)$ ，边长 $L = 120$ 。通过正等测投影显示在屏幕中心。

交互操作：

- 1/2/3 —— 绕 $x/y/z$ 轴旋转 5° （按 Shift+1/2/3 反向）；
- + / - —— 对中心放大 / 缩小 $1.05\times$ ；
- x/y/z —— 关于 $yOz / xOz / xOy$ 平面对称；
- 方向键 —— 沿 x 、 y 方向平移； PgUp/PgDn —— 沿 z 平移；
- Z —— 重置； ESC —— 退出。

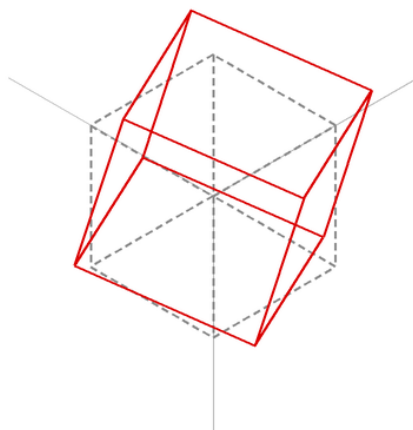


图 8: 立方体的三维变换——原始（灰色虚线）与变换后（红色实线）的正等测投影对比

观察结论：

- 立方体在初始正等测投影下，三个可见面均匀展开为正六边形轮廓中的三个菱形，证明投影矩阵推导正确；
- 绕 z 轴旋转时立方体“原地转动”，绕 x 或 y 轴旋转则会出现“前倾后仰”——三个轴的旋转分量正交，符合右手坐标系预期；

- 关于 xOy 平面对称后立方体上下翻转、关于 yOz 对称后左右翻转，验证对称矩阵正确；
- 平移与旋转的复合顺序决定最终位姿，这一点与实验六二维变换完全一致——齐次坐标的优雅在此体现得淋漓尽致。

8.6 实验收获及体会

实验八是整门课程的“集大成”——它把前面七个实验中接触到的所有思想串到了一起：

1. **光栅化（实验一、二）的最终落点。**三维立体最终绘到屏幕上时，每条边都还要靠 Bresenham 直线算法逐像素绘制；曲面则要靠中点圆/椭圆同类思想离散化。换句话说——“三维变换”产生了像素的位置”，光栅化”把像素填到屏幕上”。
2. **曲线（实验三、四）是空间造型的基础。**当几何体不是简单立方体而是曲面时，都要用 Bezier / Hermite / B 样条来表达。三维 + 曲线 + 投影 = 现代 CAD 与游戏建模的核心。
3. **填充（实验五）让线框升级为实体。**把投影后的多边形面用扫描线填充，线框立方体就变成了”实心立方体”。在此基础上加入光照与材质，就过渡到经典的 Phong / PBR 渲染。
4. **二维变换（实验六）是三维变换的”降维示例”。**从 3×3 到 4×4 ，变化的只是矩阵尺寸；底层的”齐次坐标 + 行向量右乘 + 三明治结构”思想完全一致。这种维度无关的统一让我意识到：好的抽象具有”维度无关”的鲁棒性。
5. **裁剪（实验七）的三维推广。**屏幕外的三角形要靠裁剪剔除；相机后的几何体要靠近裁剪面剔除。本实验中虽未做实际裁剪，但管线中”模型变换 $\rightarrow$ 投影 $\rightarrow$ （裁剪） $\rightarrow$ 光栅化”的整体框架已隐隐成形。

回顾八个实验，我深刻地理解到计算机图形学不是孤立的画图技巧，而是一整条以矩阵为骨架、以几何为肉身的”显示流水线”。每一个实验都对应了流水线上一个关键阶段，而矩阵这一通用语言把它们串联起来。在今后接触 OpenGL 着色器、Vulkan 渲染管线、甚至光线追踪时，本课程奠定的”几何 + 矩阵 + 离散化”思维方式都将持续为我服务。

9 实验代码

本节集中给出全部 8 个实验对应的完整源代码，按实验编号分小节。

9.1 实验一代码

按 1/2/3 键切换显示算法，按 0 键同时显示三种算法。

```

1 // =====
2 // 实验一：直线的生成 —— DDA / 中点 / Bresenham
3 // 编译: cl /EHsc exp1.cpp freeglut.lib opengl32.lib glu32.lib
4 // =====
5 #include <GL/freeglut.h>
6 #include <cmath>
7 #include <cstdio>
8 #include <algorithm>
9
10 const int W = 600, H = 600;
11 int mode = 0;          // 0:全部 1:DDA 2:中点 3:Bresenham
12 int px1 = 50, py1 = 80, px2 = 550, py2 = 420;
13
14 // ----- 工具：单像素绘制 -----
15 inline void putPixel(int x, int y) {
16     glBegin(GL_POINTS);
17     glVertex2i(x, y);
18     glEnd();
19 }
20
21 // ----- 1. DDA 算法 -----
22 void DDA_Line(int x1, int y1, int x2, int y2) {
23     int dx = x2 - x1, dy = y2 - y1;
24     int steps = std::max(std::abs(dx), std::abs(dy));
25     float xInc = (float)dx / steps;
26     float yInc = (float)dy / steps;
27     float x = (float)x1, y = (float)y1;
28     for (int i = 0; i <= steps; ++i) {
29         putPixel((int)std::lround(x), (int)std::lround(y));
30         x += xInc; y += yInc;
31     }
32 }
33

```

```
34 // ----- 2. 中点画线算法 (0 <= k <= 1) -----
35 void Midpoint_Line(int x1, int y1, int x2, int y2) {
36     int a = y1 - y2, b = x2 - x1;
37     int d = 2 * a + b;
38     int d1 = 2 * a;
39     int d2 = 2 * (a + b);
40     int x = x1, y = y1;
41     putPixel(x, y);
42     while (x < x2) {
43         if (d < 0) { ++x; ++y; d += d2; }
44         else      { ++x;      d += d1; }
45         putPixel(x, y);
46     }
47 }
48
49 // ----- 3. Bresenham 算法 (任意斜率) -----
50 void Bresenham_Line(int x1, int y1, int x2, int y2) {
51     int dx = std::abs(x2 - x1), dy = std::abs(y2 - y1);
52     int sx = (x1 < x2) ? 1 : -1;
53     int sy = (y1 < y2) ? 1 : -1;
54     int err = dx - dy;
55     int x = x1, y = y1;
56     while (true) {
57         putPixel(x, y);
58         if (x == x2 && y == y2) break;
59         int e2 = 2 * err;
60         if (e2 > -dy) { err -= dy; x += sx; }
61         if (e2 < dx) { err += dx; y += sy; }
62     }
63 }
64
65 // ----- 显示回调 -----
66 void display() {
67     glClear(GL_COLOR_BUFFER_BIT);
68     glPointSize(1.0f);
69
70     // 蓝色: DDA ( y - 10 )
71     if (mode == 0 || mode == 1) {
72         glColor3f(0.0f, 0.4f, 1.0f);
73         DDA_Line(px1, py1 - 10, px2, py2 - 10);
```

```
74     }
75     // 绿色: 中点 ( 原位 )
76     if (mode == 0 || mode == 2) {
77         glColor3f(0.0f, 0.7f, 0.0f);
78         Midpoint_Line(px1, py1, px2, py2);
79     }
80     // 红色: Bresenham ( y + 10 )
81     if (mode == 0 || mode == 3) {
82         glColor3f(1.0f, 0.0f, 0.0f);
83         Bresenham_Line(px1, py1 + 10, px2, py2 + 10);
84     }
85     glFlush();
86 }
87
88 // ----- 键盘回调 -----
89 void keyboard(unsigned char key, int, int) {
90     if (key >= '0' && key <= '3') {
91         mode = key - '0';
92         glutPostRedisplay();
93     }
94     if (key == 27) std::exit(0); // ESC 退出
95 }
96
97 // ----- 初始化 -----
98 void init() {
99     glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
100     glColor3f(0.0f, 0.0f, 0.0f);
101     glMatrixMode(GL_PROJECTION);
102     gluOrtho2D(0.0, (GLdouble)W, 0.0, (GLdouble)H);
103 }
104
105 int main(int argc, char** argv) {
106     glutInit(&argc, argv);
107     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
108     glutInitWindowSize(W, H);
109     glutInitWindowPosition(100, 100);
110     glutCreateWindow("Exp.1 Line Generation: DDA / Midpoint / Bresenham");
111     init();
112     glutDisplayFunc(display);
113     glutKeyboardFunc(keyboard);
```

```
114     glutMainLoop();
115     return 0;
116 }
```

Listing 1: 直线生成算法对比——完整源代码

9.2 实验二代码

按 + / - 键调整半径，按 [/] 键调整椭圆短半轴，按 ESC 退出。

```
1 // =====
2 // 实验二：圆弧及椭圆弧的生成 —— Bresenham 圆 / 中点圆 / 中点椭圆
3 // 编译: cl /EHsc exp2.cpp freeglut.lib opengl32.lib glu32.lib
4 // =====
5 #include <GL/freeglut.h>
6 #include <cmath>
7 #include <cstdio>
8 #include <algorithm>
9
10 const int W = 600, H = 600;
11
12 // 圆参数
13 int cx1 = 200, cy1 = 300, R = 120;
14 // 椭圆参数
15 int cx2 = 450, cy2 = 300, EA = 140, EB = 80;
16
17 // ----- 工具：单像素 -----
18 inline void putPixel(int x, int y) {
19     glBegin(GL_POINTS);
20     glVertex2i(x, y);
21     glEnd();
22 }
23
24 // ----- 圆的八分对称绘制 -----
25 inline void plot8(int xc, int yc, int x, int y) {
26     putPixel(xc + x, yc + y); putPixel(xc - x, yc + y);
27     putPixel(xc + x, yc - y); putPixel(xc - x, yc - y);
28     putPixel(xc + y, yc + x); putPixel(xc - y, yc + x);
29     putPixel(xc + y, yc - x); putPixel(xc - y, yc - x);
30 }
31
```

```
32 // ----- 椭圆的四分对称绘制 -----
33 inline void plot4(int xc, int yc, int x, int y) {
34     putPixel(xc + x, yc + y); putPixel(xc - x, yc + y);
35     putPixel(xc + x, yc - y); putPixel(xc - x, yc - y);
36 }
37
38 // ----- 1. Bresenham 圆 -----
39 void Bresenham_Circle(int xc, int yc, int r) {
40     int x = 0, y = r;
41     int p = 3 - 2 * r;
42     plot8(xc, yc, x, y);
43     while (x < y) {
44         ++x;
45         if (p < 0) {
46             p += 4 * x + 6;
47         } else {
48             --y;
49             p += 4 * (x - y) + 10;
50         }
51         plot8(xc, yc, x, y);
52     }
53 }
54
55 // ----- 2. 中点画圆 -----
56 void Midpoint_Circle(int xc, int yc, int r) {
57     int x = 0, y = r;
58     int d = 1 - r;
59     plot8(xc, yc, x, y);
60     while (x < y) {
61         if (d < 0) {
62             d += 2 * x + 3;
63         } else {
64             d += 2 * (x - y) + 5;
65             --y;
66         }
67         ++x;
68         plot8(xc, yc, x, y);
69     }
70 }
71
```

```
72 // ----- 3. 中点画椭圆 -----
73 void Midpoint_Ellipse(int xc, int yc, int a, int b) {
74     long long a2 = (long long)a * a;
75     long long b2 = (long long)b * b;
76     long long x = 0, y = b;
77     // 区域 1:  $|dy/dx| < 1$ , 沿  $x$  步进
78     long long d1 = b2 - a2 * b + a2 / 4;
79     plot4(xc, yc, (int)x, (int)y);
80     while (b2 * (x + 1) < a2 * (y - 0) - a2 / 2) { // 切换条件:  $b^2 x < a^2$ 
81         y
82         if (d1 < 0) {
83             d1 += b2 * (2 * x + 3);
84         } else {
85             d1 += b2 * (2 * x + 3) + a2 * (-2 * y + 2);
86             --y;
87         }
88         ++x;
89         plot4(xc, yc, (int)x, (int)y);
90     }
91     // 区域 2:  $|dy/dx| \geq 1$ , 沿  $y$  步进
92     long long d2 = b2 * (2*x + 1) * (2*x + 1) / 4
93         + a2 * (y - 1) * (y - 1) - a2 * b2;
94     while (y > 0) {
95         if (d2 < 0) {
96             d2 += b2 * (2 * x + 2) + a2 * (-2 * y + 3);
97             ++x;
98         } else {
99             d2 += a2 * (-2 * y + 3);
100         }
101         --y;
102         plot4(xc, yc, (int)x, (int)y);
103     }
104 }
105 // ----- 显示回调 -----
106 void display() {
107     glClear(GL_COLOR_BUFFER_BIT);
108     glPointSize(1.0f);
109
110     // 红色: Bresenham 圆 (外侧)
```

```
111     glColor3f(1.0f, 0.0f, 0.0f);
112     Bresenham_Circle(cx1, cy1, R);
113
114     // 绿色: 中点圆 (内偏 2 像素以便观察)
115     glColor3f(0.0f, 0.7f, 0.0f);
116     Midpoint_Circle(cx1, cy1, R - 2);
117
118     // 蓝色: 中点椭圆
119     glColor3f(0.0f, 0.0f, 1.0f);
120     Midpoint_Ellipse(cx2, cy2, EA, EB);
121
122     glFlush();
123 }
124
125 // ----- 键盘回调 -----
126 void keyboard(unsigned char key, int, int) {
127     switch (key) {
128         case '+': case '=': R += 5; break;
129         case '-': case '_': R = std::max(10, R - 5); break;
130         case ']':      EB += 5; break;
131         case '[':      EB = std::max(10, EB - 5); break;
132         case 27: std::exit(0);
133     }
134     glutPostRedisplay();
135 }
136
137 void init() {
138     glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
139     glColor3f(0.0f, 0.0f, 0.0f);
140     glMatrixMode(GL_PROJECTION);
141     gluOrtho2D(0.0, (GLdouble)W, 0.0, (GLdouble)H);
142 }
143
144 int main(int argc, char** argv) {
145     glutInit(&argc, argv);
146     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
147     glutInitWindowSize(W, H);
148     glutInitWindowPosition(100, 100);
149     glutCreateWindow("Exp.2 Circle & Ellipse: Bresenham / Midpoint");
150     init();
```

```

151     glutDisplayFunc(display);
152     glutKeyboardFunc(keyboard);
153     glutMainLoop();
154     return 0;
155 }

```

Listing 2: 圆弧/椭圆生成——完整源代码

9.3 实验三代码

程序同时绘制三种 Bezier 曲线：左上为二次 Bezier、右上为三次 Bezier、下方为两段拼接的 C^1 连续分段三次 Bezier。控制多边形以蓝色虚线呈现，曲线本身以红色实线呈现。

```

1  // =====
2  // 实验三: Bezier 曲线生成
3  // - 二次 Bezier (直接公式)
4  // - 三次 Bezier (直接公式 / de Casteljau)
5  // - 分段三次 Bezier ( $C^1$  连续)
6  // 编译: cl /EHsc exp3.cpp freeglut.lib opengl32.lib glu32.lib
7  // =====
8  #include <GL/freeglut.h>
9  #include <vector>
10 #include <cstdio>
11 #include <cstdlib>
12
13 const int W = 700, H = 500;
14
15 struct P { float x, y; };
16
17 // 二次 Bezier 控制点 (左上)
18 P quad[3] = { {50, 400}, {200, 80}, {350, 400} };
19
20 // 三次 Bezier 控制点 (右上)
21 P cubic[4] = { {380, 400}, {450, 80}, {580, 80}, {650, 400} };
22
23 // 分段三次 Bezier 控制点 (下方), 第二段  $Q1$  由  $C^1$  条件自动得出
24 P seg1[4] = { {80, 100}, {180, 280}, {280, 280}, {380, 100} };
25 P seg2[4]; // 在 init 中用  $C^1$  条件填入
26
27 // ----- 二次 Bezier 直接公式 -----

```

```
28 P bezier2(float t, const P* p) {
29     float u = 1 - t;
30     P r;
31     r.x = u*u*p[0].x + 2*u*t*p[1].x + t*t*p[2].x;
32     r.y = u*u*p[0].y + 2*u*t*p[1].y + t*t*p[2].y;
33     return r;
34 }
35
36 // ----- 三次 Bezier 直接公式 -----
37 P bezier3(float t, const P* p) {
38     float u = 1 - t;
39     float b0 = u*u*u, b1 = 3*u*u*t, b2 = 3*u*t*t, b3 = t*t*t;
40     P r;
41     r.x = b0*p[0].x + b1*p[1].x + b2*p[2].x + b3*p[3].x;
42     r.y = b0*p[0].y + b1*p[1].y + b2*p[2].y + b3*p[3].y;
43     return r;
44 }
45
46 // ----- de Casteljau (迭代版) -----
47 P deCasteljau(float t, const P* ctrl, int n) {
48     std::vector<P> tmp(ctrl, ctrl + n + 1);
49     for (int k = 1; k <= n; ++k) {
50         for (int i = 0; i <= n - k; ++i) {
51             tmp[i].x = (1 - t) * tmp[i].x + t * tmp[i + 1].x;
52             tmp[i].y = (1 - t) * tmp[i].y + t * tmp[i + 1].y;
53         }
54     }
55     return tmp[0];
56 }
57
58 // ----- 用直线段连接采样点的方式画曲线 -----
59 void drawCurve(P (*func)(float, const P*), const P* ctrl, int N) {
60     glBegin(GL_LINE_STRIP);
61     for (int i = 0; i <= N; ++i) {
62         float t = (float)i / N;
63         P pt = func(t, ctrl);
64         glVertex2f(pt.x, pt.y);
65     }
66     glEnd();
67 }
```

```
68
69 // ----- 画控制多边形 (虚线) -----
70 void drawControlPolygon(const P* ctrl, int n) {
71     glEnable(GL_LINE_STIPPLE);
72     glLineStipple(1, 0x00FF);
73     glBegin(GL_LINE_STRIP);
74     for (int i = 0; i <= n; ++i) glVertex2f(ctrl[i].x, ctrl[i].y);
75     glEnd();
76     glDisable(GL_LINE_STIPPLE);
77     // 控制点
78     glPointSize(6.0f);
79     glBegin(GL_POINTS);
80     for (int i = 0; i <= n; ++i) glVertex2f(ctrl[i].x, ctrl[i].y);
81     glEnd();
82 }
83
84 void display() {
85     glClear(GL_COLOR_BUFFER_BIT);
86     glLineWidth(2.0f);
87
88     // 蓝色虚线: 控制多边形
89     glColor3f(0.0f, 0.0f, 1.0f);
90     drawControlPolygon(quad, 2);
91     drawControlPolygon(cubic, 3);
92     drawControlPolygon(seg1, 3);
93     drawControlPolygon(seg2, 3);
94
95     // 红色实线: Bezier 曲线
96     glColor3f(1.0f, 0.0f, 0.0f);
97     drawCurve(bezier2, quad, 200);
98     drawCurve(bezier3, cubic, 200);
99     drawCurve(bezier3, seg1, 200);
100    drawCurve(bezier3, seg2, 200);
101
102    glFlush();
103 }
104
105 void init() {
106     glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
107     glEnable(GL_LINE_SMOOTH);
```

```

108     glHint(GL_LINE_SMOOTH_HINT, GL_NICEST);
109     glEnable(GL_BLEND);
110     glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
111     glMatrixMode(GL_PROJECTION);
112     gluOrtho2D(0.0, (GLdouble)W, 0.0, (GLdouble)H);
113
114     // 由 seg1 末两点用  $C^1$  条件构造 seg2
115     seg2[0] = seg1[3]; //  $P3 = Q0$ 
116     seg2[1].x = 2 * seg1[3].x - seg1[2].x; //  $Q1 = 2*P3 - P2$ 
117     seg2[1].y = 2 * seg1[3].y - seg1[2].y;
118     seg2[2].x = 580; seg2[2].y = 280;
119     seg2[3].x = 660; seg2[3].y = 100;
120 }
121
122 int main(int argc, char** argv) {
123     glutInit(&argc, argv);
124     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
125     glutInitWindowSize(W, H);
126     glutInitWindowPosition(100, 100);
127     glutCreateWindow("Exp.3 Bezier Curves: Quadratic / Cubic / Piecewise
128          $C^1$ ");
129     init();
130     glutDisplayFunc(display);
131     glutMainLoop();
132     return 0;
133 }

```

Listing 3: Bezier 曲线生成——完整源代码

9.4 实验四代码

程序同屏显示三种 Hermite 曲线：左上为二次、右上为三次、下方为三段拼接的 C^1 连续曲线。切向量以绿色箭头呈现。按 +/- 键可同步缩放三次 Hermite 的切向量长度，观察曲线形状变化。

```

1 // =====
2 // 实验四: Hermite 曲线生成
3 // - 二次 Hermite (端点 + 起点切向)
4 // - 三次 Hermite (端点 + 两端切向)
5 // - 分段三次 Hermite (共享切向 +  $C^1$  连续)
6 // 编译: cl /EHsc exp4.cpp freeglut.lib opengl32.lib glu32.lib

```

```

7  // =====
8  #include <GL/freeglut.h>
9  #include <vector>
10 #include <cmath>
11 #include <cstdlib>
12
13 const int W = 700, H = 500;
14 float scale = 1.0f; // 三次 Hermite 切向量长度的全局缩放系数
15
16 struct V { float x, y; };
17
18 // ----- 二次 Hermite :  $(1-t^2)P_0 + t^2P_1 + t(1-t)R_0$  -----
19 V hermite2(float t, V P0, V P1, V R0) {
20     float t2 = t * t;
21     V r;
22     r.x = (1 - t2) * P0.x + t2 * P1.x + t * (1 - t) * R0.x;
23     r.y = (1 - t2) * P0.y + t2 * P1.y + t * (1 - t) * R0.y;
24     return r;
25 }
26
27 // ----- 三次 Hermite : 标准 4 基函数 -----
28 V hermite3(float t, V P0, V P1, V R0, V R1) {
29     float t2 = t * t, t3 = t2 * t;
30     float h0 = 2*t3 - 3*t2 + 1;
31     float h1 = -2*t3 + 3*t2;
32     float h2 = t3 - 2*t2 + t;
33     float h3 = t3 - t2;
34     V r;
35     r.x = h0*P0.x + h1*P1.x + h2*R0.x + h3*R1.x;
36     r.y = h0*P0.y + h1*P1.y + h2*R0.y + h3*R1.y;
37     return r;
38 }
39
40 // ----- 工具: 画曲线 / 端点 / 切向量箭头 -----
41 void drawSampledCurve(std::vector<V>& pts) {
42     glBegin(GL_LINE_STRIP);
43     for (auto& p : pts) glVertex2f(p.x, p.y);
44     glEnd();
45 }
46

```

```
47 void drawDot(V p) {
48     glPointSize(8.0f);
49     glBegin(GL_POINTS);
50     glVertex2f(p.x, p.y);
51     glEnd();
52 }
53
54 void drawArrow(V from, V dir) {
55     V to = { from.x + dir.x, from.y + dir.y };
56     // 主干: 绿色虚线
57     glEnable(GL_LINE_STIPPLE);
58     glLineStipple(1, 0x00FF);
59     glBegin(GL_LINES);
60     glVertex2f(from.x, from.y);
61     glVertex2f(to.x, to.y);
62     glEnd();
63     glDisable(GL_LINE_STIPPLE);
64     // 箭头: 等腰小三角
65     float len = std::sqrt(dir.x*dir.x + dir.y*dir.y);
66     if (len < 1e-3f) return;
67     float ux = dir.x / len, uy = dir.y / len;
68     float ah = 10.0f; // 箭头长
69     float aw = 5.0f; // 箭头半宽
70     V a1 = { to.x - ah*ux + aw*(-uy), to.y - ah*uy + aw*( ux) };
71     V a2 = { to.x - ah*ux - aw*(-uy), to.y - ah*uy - aw*( ux) };
72     glBegin(GL_TRIANGLES);
73     glVertex2f(to.x, to.y);
74     glVertex2f(a1.x, a1.y);
75     glVertex2f(a2.x, a2.y);
76     glEnd();
77 }
78
79 void display() {
80     glClear(GL_COLOR_BUFFER_BIT);
81     glLineWidth(2.0f);
82
83     // ----- 二次 Hermite (左上) -----
84     V P0a = {60, 400}, P1a = {330, 400}, R0a = {120, -400};
85     std::vector<V> curveA;
86     for (int i = 0; i <= 200; ++i)
```

```

87     curveA.push_back(hermite2((float)i / 200, P0a, P1a, R0a));
88     glColor3f(1, 0, 0); drawSampledCurve(curveA);
89     glColor3f(0, 0, 0); drawDot(P0a); drawDot(P1a);
90     glColor3f(0, 0.7f, 0); drawArrow(P0a, R0a);
91
92     // ----- 三次 Hermite (右上) -----
93     V P0b = {380, 400}, P1b = {660, 400};
94     V R0b = {200*scale, -400*scale}, R1b = {-200*scale, -400*scale};
95     std::vector<V> curveB;
96     for (int i = 0; i <= 200; ++i)
97         curveB.push_back(hermite3((float)i / 200, P0b, P1b, R0b, R1b));
98     glColor3f(1, 0, 0); drawSampledCurve(curveB);
99     glColor3f(0, 0, 0); drawDot(P0b); drawDot(P1b);
100    glColor3f(0, 0.7f, 0); drawArrow(P0b, R0b); drawArrow(P1b, R1b);
101
102    // ----- 分段三次 Hermite (下方),  $C^1$  连续: 相邻段共享切向 -----
103    V S0 = { 60, 100}, S1 = {220, 220}, S2 = {380, 100}, S3 = {550, 220},
104        S4 = {680, 80};
105    V T0 = {200, 100}, T1 = { 0, 200}, T2 = {200,-100}, T3 = {200, 200},
106        T4 = {-100,-200};
107    std::vector<V> seg;
108    auto addSeg = [&](V A, V B, V Ra, V Rb) {
109        for (int i = 0; i <= 100; ++i)
110            seg.push_back(hermite3((float)i / 100, A, B, Ra, Rb));
111    };
112    addSeg(S0, S1, T0, T1);
113    addSeg(S1, S2, T1, T2); // T1 与 T2 共享起点切向 →  $C^0 + C^1$ 
114    addSeg(S2, S3, T2, T3);
115    addSeg(S3, S4, T3, T4);
116    glColor3f(1, 0, 0); drawSampledCurve(seg);
117    glColor3f(0, 0, 0);
118    drawDot(S0); drawDot(S1); drawDot(S2); drawDot(S3); drawDot(S4);
119
120    glFlush();
121 }
122
123 void keyboard(unsigned char key, int, int) {
124     if (key == '+' || key == '=') scale *= 1.2f;
125     if (key == '-' || key == '_') scale /= 1.2f;
126     if (key == 27) std::exit(0);

```

```

125     glutPostRedisplay();
126 }
127
128 void init() {
129     glClearColor(1, 1, 1, 1);
130     glEnable(GL_LINE_SMOOTH);
131     glHint(GL_LINE_SMOOTH_HINT, GL_NICEST);
132     glEnable(GL_BLEND);
133     glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
134     glMatrixMode(GL_PROJECTION);
135     gluOrtho2D(0.0, (GLdouble)W, 0.0, (GLdouble)H);
136 }
137
138 int main(int argc, char** argv) {
139     glutInit(&argc, argv);
140     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
141     glutInitWindowSize(W, H);
142     glutInitWindowPosition(100, 100);
143     glutCreateWindow("Exp.4 Hermite Curves: Quadratic / Cubic / Piecewise
144                      C~1");
145     init();
146     glutDisplayFunc(display);
147     glutKeyboardFunc(keyboard);
148     glutMainLoop();
149     return 0;
150 }

```

Listing 4: Hermite 曲线生成——完整源代码

9.5 实验五代码

程序在内存里维护一块 $H \times W \times 3$ 的画布, 先用 Bresenham 绘出 3 个相同的飞镖形凹多边形, 再分别用 4-连通种子、8-连通种子、扫描线填充上色, 最后通过 `glDrawPixels` 一次性显示。

```

1 // =====
2 // 实验五: 多边形区域填充 —— 4 连通 / 8 连通种子填充 / 扫描线填充
3 // 编译: cl /EHsc exp5.cpp freeglut.lib opengl32.lib glu32.lib
4 // =====
5 #include <GL/freeglut.h>
6 #include <vector>

```

```
7  #include <stack>
8  #include <list>
9  #include <algorithm>
10 #include <cstring>
11
12 const int W = 720, H = 480;
13 unsigned char canvas[H][W][3];
14
15 struct P { int x, y; };
16 struct C { unsigned char r, g, b; };
17
18 const C BLACK = {0, 0, 0};
19 const C BLUE = {30, 144, 255};
20 const C GREEN = {30, 200, 60};
21 const C ORANGE = {255, 140, 0};
22 const C WHITE = {255, 255, 255};
23
24 // ----- 像素工具 -----
25 inline void setPx(int x, int y, C c) {
26     if (x < 0 || x >= W || y < 0 || y >= H) return;
27     canvas[y][x][0] = c.r;
28     canvas[y][x][1] = c.g;
29     canvas[y][x][2] = c.b;
30 }
31 inline C getPx(int x, int y) {
32     return { canvas[y][x][0], canvas[y][x][1], canvas[y][x][2] };
33 }
34 inline bool eqColor(C a, C b) {
35     return a.r == b.r && a.g == b.g && a.b == b.b;
36 }
37
38 // ----- Bresenham 直线 (用作多边形边界绘制) -----
39 void drawLine(int x1, int y1, int x2, int y2, C c) {
40     int dx = std::abs(x2 - x1), dy = std::abs(y2 - y1);
41     int sx = (x1 < x2) ? 1 : -1;
42     int sy = (y1 < y2) ? 1 : -1;
43     int err = dx - dy;
44     while (true) {
45         setPx(x1, y1, c);
46         if (x1 == x2 && y1 == y2) break;
```

```
47     int e2 = 2 * err;
48     if (e2 > -dy) { err -= dy; x1 += sx; }
49     if (e2 < dx) { err += dx; y1 += sy; }
50 }
51 }
52
53 void drawPolygon(const std::vector<P>& v, C c) {
54     int n = (int)v.size();
55     for (int i = 0; i < n; ++i) {
56         const P& a = v[i];
57         const P& b = v[(i + 1) % n];
58         drawLine(a.x, a.y, b.x, b.y, c);
59     }
60 }
61
62 // ----- 1. 4 连通种子填充 -----
63 void seedFill4(int sx, int sy, C fill, C border) {
64     std::stack<P> S;
65     S.push({sx, sy});
66     while (!S.empty()) {
67         P p = S.top(); S.pop();
68         if (p.x < 0 || p.x >= W || p.y < 0 || p.y >= H) continue;
69         C cur = getPx(p.x, p.y);
70         if (eqColor(cur, border) || eqColor(cur, fill)) continue;
71         setPx(p.x, p.y, fill);
72         S.push({p.x + 1, p.y});
73         S.push({p.x - 1, p.y});
74         S.push({p.x, p.y + 1});
75         S.push({p.x, p.y - 1});
76     }
77 }
78
79 // ----- 2. 8 连通种子填充 -----
80 void seedFill8(int sx, int sy, C fill, C border) {
81     std::stack<P> S;
82     S.push({sx, sy});
83     while (!S.empty()) {
84         P p = S.top(); S.pop();
85         if (p.x < 0 || p.x >= W || p.y < 0 || p.y >= H) continue;
86         C cur = getPx(p.x, p.y);
```

```

87     if (eqColor(cur, border) || eqColor(cur, fill)) continue;
88     setPx(p.x, p.y, fill);
89     for (int dx = -1; dx <= 1; ++dx)
90         for (int dy = -1; dy <= 1; ++dy)
91             if (dx || dy) S.push({p.x + dx, p.y + dy});
92     }
93 }
94
95 // ----- 3. 扫描线填充 (基于 ET / AET) -----
96 struct Edge {
97     int    yMax;    // 上端点 y
98     double x;      // 当前扫描线对应交点 x
99     double dx;      //  $1/k = (x_{top} - x_{bot}) / (y_{top} - y_{bot})$ 
100 };
101
102 void scanLineFill(const std::vector<P>& poly, C fill) {
103     int n = (int)poly.size();
104     int yMin = H, yMax = 0;
105     for (auto& p : poly) {
106         yMin = std::min(yMin, p.y);
107         yMax = std::max(yMax, p.y);
108     }
109     // 边表 ET: 以 yMin 为索引 (相对于 yMin 偏移)
110     std::vector<std::list<Edge>> ET(yMax - yMin + 1);
111     for (int i = 0; i < n; ++i) {
112         P a = poly[i], b = poly[(i + 1) % n];
113         if (a.y == b.y) continue;           // 跳过水平边
114         if (a.y > b.y) std::swap(a, b);     // a 为下端点
115         Edge e;
116         e.yMax = b.y;
117         e.x    = a.x;
118         e.dx   = (double)(b.x - a.x) / (b.y - a.y);
119         ET[a.y - yMin].push_back(e);
120     }
121     // 主扫描循环
122     std::list<Edge> AET;
123     for (int y = yMin; y < yMax; ++y) {
124         // 1. 入 AET
125         for (auto& e : ET[y - yMin]) AET.push_back(e);
126         // 2. 删除 yMax == y 的边

```

```

127     AET.remove_if([&](const Edge& e) { return e.yMax == y; });
128     // 3. 按 x 排序
129     AET.sort([](const Edge& a, const Edge& b) { return a.x < b.x; });
130     // 4. 两两配对填充
131     auto it = AET.begin();
132     while (it != AET.end()) {
133         auto j = std::next(it);
134         if (j == AET.end()) break;
135         int xL = (int)std::ceil(it->x);
136         int xR = (int)std::floor(j->x);
137         for (int x = xL; x <= xR; ++x) setPx(x, y, fill);
138         it = std::next(j);
139     }
140     // 5. 更新 x
141     for (auto& e : AET) e.x += e.dx;
142 }
143 }
144
145 // ----- 主体：三个相同凹多边形 -----
146 std::vector<P> makeDart(int cx, int cy) {
147     return {
148         {cx,      cy + 110},
149         {cx + 30, cy + 30 },
150         {cx + 100, cy + 0 },
151         {cx + 30, cy - 30 },
152         {cx,      cy - 110},
153         {cx - 30, cy - 30 },
154         {cx - 100, cy + 0 },
155         {cx - 30, cy + 30 }
156     };
157 }
158
159 void prepareScene() {
160     std::memset(canvas, 255, sizeof canvas); // 白底
161     auto poly1 = makeDart(120, 240);
162     auto poly2 = makeDart(360, 240);
163     auto poly3 = makeDart(600, 240);
164
165     drawPolygon(poly1, BLACK);
166     drawPolygon(poly2, BLACK);

```

```

167     drawPolygon(poly3, BLACK);
168
169     seedFill14 (120, 240, BLUE, BLACK);
170     seedFill18 (360, 240, GREEN, BLACK);
171     scanLineFill(poly3, ORANGE);
172 }
173
174 void display() {
175     glClear(GL_COLOR_BUFFER_BIT);
176     glDrawPixels(W, H, GL_RGB, GL_UNSIGNED_BYTE, canvas);
177     glFlush();
178 }
179
180 void init() {
181     glClearColor(1, 1, 1, 1);
182     glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
183     glMatrixMode(GL_PROJECTION);
184     gluOrtho2D(0, W, 0, H);
185     prepareScene();
186 }
187
188 int main(int argc, char** argv) {
189     glutInit(&argc, argv);
190     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
191     glutInitWindowSize(W, H);
192     glutInitWindowPosition(100, 100);
193     glutCreateWindow("Exp.5 Polygon Filling: 4-Conn / 8-Conn / Scan-Line");
194     init();
195     glutDisplayFunc(display);
196     glutMainLoop();
197     return 0;
198 }

```

Listing 5: 多边形区域填充——完整源代码

9.6 实验六代码

程序原始顶点表保存在数组 `star_orig` 中不被修改，每次按键事件只更新累积变换矩阵 `M`，显示时再统一变换原始顶点，避免反复浮点变换造成失真。

```

1 // =====

```

```
2 // 实验六：二维几何变换 —— 平移 / 旋转 / 缩放 / 对称 + 鼠标拾取
3 // 编译：cl /EHsc exp6.cpp freeglut.lib opengl32.lib glu32.lib
4 // =====
5 #include <GL/freeglut.h>
6 #include <cmath>
7 #include <cstdlib>
8 #include <vector>
9
10 const int W = 720, H = 540;
11 const double PI = 3.14159265358979323846;
12
13 // 3x3 矩阵（行优先）—— 行向量右乘约定
14 struct M3 { double m[3][3]; };
15
16 M3 mIdentity() {
17     M3 r{};
18     r.m[0][0] = r.m[1][1] = r.m[2][2] = 1;
19     return r;
20 }
21 M3 mMul(const M3& A, const M3& B) { // A * B
22     M3 r{};
23     for (int i = 0; i < 3; ++i)
24         for (int j = 0; j < 3; ++j)
25             for (int k = 0; k < 3; ++k)
26                 r.m[i][j] += A.m[i][k] * B.m[k][j];
27     return r;
28 }
29 M3 mTranslate(double tx, double ty) {
30     M3 r = mIdentity();
31     r.m[2][0] = tx; r.m[2][1] = ty;
32     return r;
33 }
34 M3 mRotate(double deg) {
35     double t = deg * PI / 180.0;
36     M3 r = mIdentity();
37     double c = std::cos(t), s = std::sin(t);
38     r.m[0][0] = c; r.m[0][1] = s;
39     r.m[1][0] = -s; r.m[1][1] = c;
40     return r;
41 }
```

```

42 M3 mScale(double sx, double sy) {
43     M3 r = mIdentity();
44     r.m[0][0] = sx; r.m[1][1] = sy;
45     return r;
46 }
47 M3 mReflect(char axis) { // 'x', 'y', 'o', 'd'(y=x)
48     M3 r = mIdentity();
49     if (axis == 'x') r.m[1][1] = -1;
50     if (axis == 'y') r.m[0][0] = -1;
51     if (axis == 'o') { r.m[0][0] = -1; r.m[1][1] = -1; }
52     if (axis == 'd') { r.m[0][0] = 0; r.m[0][1] = 1; r.m[1][0] = 1;
        r.m[1][1] = 0; }
53     return r;
54 }
55
56 // 应用变换:  $\begin{bmatrix} x & y & 1 \end{bmatrix} * M$ 
57 void applyM(const M3& M, double x, double y, double& xo, double& yo) {
58     xo = x * M.m[0][0] + y * M.m[1][0] + M.m[2][0];
59     yo = x * M.m[0][1] + y * M.m[1][1] + M.m[2][1];
60 }
61
62 // 五角星 10 个顶点 (外/内交替): 构造在 origin, 半径 R/r
63 std::vector<std::pair<double, double>> makeStar() {
64     std::vector<std::pair<double, double>> v;
65     double Ro = 120, Ri = 50;
66     for (int i = 0; i < 10; ++i) {
67         double a = -PI / 2 + i * PI / 5; // 顶尖朝上
68         double rr = (i & 1) ? Ri : Ro;
69         v.push_back({rr * std::cos(a), rr * std::sin(a)});
70     }
71     return v;
72 }
73
74 const auto star_orig = makeStar();
75 M3 curM = mIdentity(); // 累积变换
76 double cx = 360, cy = 270; // 当前图形中心 (初值 = 屏幕中心)
77 bool dragging = false;
78 int lastMx, lastMy;
79
80 // "绕中心 C" 的三明治:  $T(-C) * M_{basic} * T(C)$ 

```

```
81 M3 aroundCenter(const M3& Mb) {
82     return mMul(mMul(mTranslate(-cx, -cy), Mb), mTranslate(cx, cy));
83 }
84
85 // ----- 计算变换后顶点 -----
86 std::vector<std::pair<double,double>> transformed() {
87     std::vector<std::pair<double,double>> r;
88     r.reserve(star_orig.size());
89     for (auto& p : star_orig) {
90         double xo, yo;
91         // 原始坐标先平移到 (cx,cy) 再应用 curM
92         applyM(curM, p.first + cx, p.second + cy, xo, yo);
93         r.push_back({xo, yo});
94     }
95     return r;
96 }
97
98 // ----- 多边形绘制（线框） -----
99 void drawPoly(const std::vector<std::pair<double,double>>& v, bool dashed)
100 {
101     if (dashed) { glEnable(GL_LINE_STIPPLE); glLineStipple(1, 0x00FF); }
102     glBegin(GL_LINE_LOOP);
103     for (auto& p : v) glVertex2d(p.first, p.second);
104     glEnd();
105     if (dashed) glDisable(GL_LINE_STIPPLE);
106 }
107
108 // ----- 点在多边形内（射线-奇偶法，上闭下开） -----
109 bool inside(const std::vector<std::pair<double,double>>& v, double px,
110             double py) {
111     int n = (int)v.size(), cnt = 0;
112     for (int i = 0; i < n; ++i) {
113         auto a = v[i], b = v[(i + 1) % n];
114         if ((a.second <= py) == (b.second <= py)) continue; // 上闭下开
115         double xCross = a.first + (py - a.second) * (b.first - a.first) /
116             (b.second - a.second);
117         if (xCross > px) ++cnt;
118     }
119     return cnt & 1;
120 }
```

```
118
119 // ----- 显示 -----
120 void display() {
121     glClear(GL_COLOR_BUFFER_BIT);
122     glLineWidth(2.0f);
123
124     // 原始图形（灰色虚线，固定在屏幕中心）
125     std::vector<std::pair<double,double>> ref;
126     for (auto& p : star_orig)
127         ref.push_back({p.first + 360, p.second + 270});
128     glColor3f(0.6f, 0.6f, 0.6f);
129     drawPoly(ref, true);
130
131     // 变换后图形（红色实线）
132     auto cur = transformed();
133     glColor3f(1.0f, 0.0f, 0.0f);
134     drawPoly(cur, false);
135
136     glFlush();
137 }
138
139 // ----- 键盘 -----
140 void keyboard(unsigned char key, int, int) {
141     M3 base;
142     switch (key) {
143         case 'R': base = mRotate(-15); curM = mMul(curM,
144             aroundCenter(base)); break;
145         case 'r': base = mRotate( 15); curM = mMul(curM,
146             aroundCenter(base)); break;
147         case '+': case '=': base = mScale(1.1, 1.1); curM = mMul(curM,
148             aroundCenter(base)); break;
149         case '-': case '_': base = mScale(1/1.1, 1/1.1); curM = mMul(curM,
150             aroundCenter(base)); break;
151         case 'x': base = mReflect('x'); curM = mMul(curM,
152             aroundCenter(base)); break;
153         case 'y': base = mReflect('y'); curM = mMul(curM,
154             aroundCenter(base)); break;
155         case 'o': base = mReflect('o'); curM = mMul(curM,
156             aroundCenter(base)); break;
157         case 'Z': curM = mIdentity(); cx = 360; cy = 270;
```

```
        break;
151     case 27 : std::exit(0);
152     default : return;
153 }
154 glutPostRedisplay();
155 }
156
157 // ----- 鼠标（拾取 + 拖动平移） -----
158 void mouse(int button, int state, int mx, int my) {
159     if (button != GLUT_LEFT_BUTTON) return;
160     int wy = H - my; // OpenGL y 朝上
161     if (state == GLUT_DOWN) {
162         auto cur = transformed();
163         if (inside(cur, mx, wy)) { dragging = true; lastMx = mx; lastMy =
            wy; }
164     } else {
165         dragging = false;
166     }
167 }
168 void motion(int mx, int my) {
169     if (!dragging) return;
170     int wy = H - my;
171     double dx = mx - lastMx, dy = wy - lastMy;
172     cx += dx; cy += dy;
173     lastMx = mx; lastMy = wy;
174     glutPostRedisplay();
175 }
176
177 void init() {
178     glClearColor(1, 1, 1, 1);
179     glEnable(GL_LINE_SMOOTH);
180     glHint(GL_LINE_SMOOTH_HINT, GL_NICEST);
181     glEnable(GL_BLEND);
182     glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
183     glMatrixMode(GL_PROJECTION);
184     gluOrtho2D(0, W, 0, H);
185 }
186
187 int main(int argc, char** argv) {
188     glutInit(&argc, argv);
```

```

189     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
190     glutInitWindowSize(W, H);
191     glutInitWindowPosition(100, 100);
192     glutCreateWindow("Exp.6 2D Geometric Transformations");
193     init();
194     glutDisplayFunc(display);
195     glutKeyboardFunc(keyboard);
196     glutMouseFunc(mouse);
197     glutMotionFunc(motion);
198     glutMainLoop();
199     return 0;
200 }

```

Listing 6: 二维几何变换——完整源代码

9.7 实验七代码

程序在窗口中绘制一个深灰色虚线矩形作为裁剪窗口，预置 6 条测试线段（覆盖 5 种典型情形）和一个凹多边形。按 w/s/a/d 调节裁剪窗口大小，按 ESC 退出。

```

1  // =====
2  // 实验七：裁剪算法 —— Cohen-Sutherland + Sutherland-Hodgman
3  // 编译: cl /EHsc exp7.cpp freeglut.lib opengl32.lib glu32.lib
4  // =====
5  #include <GL/freeglut.h>
6  #include <vector>
7  #include <cstdlib>
8
9  const int W = 720, H = 540;
10
11 struct V { double x, y; };
12
13 // 裁剪窗口
14 double xL = 180, xR = 540, yB = 150, yT = 420;
15
16 // ----- Cohen-Sutherland 编码 -----
17 const int LEFT_C = 1, RIGHT_C = 2, BOT_C = 4, TOP_C = 8;
18 int encode(double x, double y) {
19     int c = 0;
20     if (x < xL) c |= LEFT_C;
21     if (x > xR) c |= RIGHT_C;

```

```

22     if (y < yB) c |= BOT_C;
23     if (y > yT) c |= TOP_C;
24     return c;
25 }
26
27 // ----- Cohen-Sutherland 直线裁剪 -----
28 bool cohenSutherland(V& A, V& B) {
29     int cA = encode(A.x, A.y);
30     int cB = encode(B.x, B.y);
31     while (true) {
32         if ((cA | cB) == 0) return true;    // "取"
33         if ((cA & cB) != 0) return false;   // "弃"
34         int cOut = cA ? cA : cB;
35         double x = 0, y = 0;
36         if (cOut & TOP_C) { x = A.x + (B.x - A.x) * (yT - A.y) / (B.y -
            A.y); y = yT; }
37         else if (cOut & BOT_C) { x = A.x + (B.x - A.x) * (yB - A.y) / (B.y
            - A.y); y = yB; }
38         else if (cOut & RIGHT_C) { y = A.y + (B.y - A.y) * (xR - A.x) /
            (B.x - A.x); x = xR; }
39         else if (cOut & LEFT_C) { y = A.y + (B.y - A.y) * (xL - A.x) / (B.x
            - A.x); x = xL; }
40         if (cOut == cA) { A = {x, y}; cA = encode(x, y); }
41         else { B = {x, y}; cB = encode(x, y); }
42     }
43 }
44
45 // ----- Sutherland-Hodgman 单边裁剪 -----
46 // edge = 0:Left, 1:Right, 2:Bottom, 3:Top
47 bool inside(V p, int edge) {
48     switch (edge) {
49         case 0: return p.x >= xL;
50         case 1: return p.x <= xR;
51         case 2: return p.y >= yB;
52         case 3: return p.y <= yT;
53     }
54     return false;
55 }
56 V intersect(V A, V B, int edge) {
57     V r{};

```

```

58     double t = 0;
59     switch (edge) {
60         case 0: t = (xL - A.x) / (B.x - A.x); r.x = xL; r.y = A.y + t *
                (B.y - A.y); break;
61         case 1: t = (xR - A.x) / (B.x - A.x); r.x = xR; r.y = A.y + t *
                (B.y - A.y); break;
62         case 2: t = (yB - A.y) / (B.y - A.y); r.y = yB; r.x = A.x + t *
                (B.x - A.x); break;
63         case 3: t = (yT - A.y) / (B.y - A.y); r.y = yT; r.x = A.x + t *
                (B.x - A.x); break;
64     }
65     return r;
66 }
67 std::vector<V> clipEdge(const std::vector<V>& in, int edge) {
68     std::vector<V> out;
69     int n = (int)in.size();
70     if (n == 0) return out;
71     for (int i = 0; i < n; ++i) {
72         V cur = in[i];
73         V nxt = in[(i + 1) % n];
74         bool ci = inside(cur, edge), ni = inside(nxt, edge);
75         if (ci && ni) out.push_back(nxt);
76         else if (ci && !ni) out.push_back(intersect(cur, nxt, edge));
77         else if (!ci && ni){ out.push_back(intersect(cur, nxt, edge));
78                             out.push_back(nxt); }
79         // else: 两点都在外, 无输出
80     }
81     return out;
82 }
83
84 std::vector<V> sutherlandHodgman(std::vector<V> poly) {
85     for (int e = 0; e < 4; ++e) poly = clipEdge(poly, e);
86     return poly;
87 }
88
89 // ----- 绘制工具 -----
90 void drawLine(V A, V B) {
91     glBegin(GL_LINES);
92     glVertex2d(A.x, A.y);
93     glVertex2d(B.x, B.y);

```

```
94     glEnd();
95 }
96 void drawPoly(const std::vector<V>& v) {
97     if (v.empty()) return;
98     glBegin(GL_LINE_LOOP);
99     for (auto& p : v) glVertex2d(p.x, p.y);
100    glEnd();
101 }
102 void drawClipWindow() {
103     glColor3f(0.4f, 0.4f, 0.4f);
104     glEnable(GL_LINE_STIPPLE);
105     glLineStipple(1, 0x00FF);
106     glBegin(GL_LINE_LOOP);
107     glVertex2d(xL, yB); glVertex2d(xR, yB);
108     glVertex2d(xR, yT); glVertex2d(xL, yT);
109     glEnd();
110     glDisable(GL_LINE_STIPPLE);
111 }
112
113 // ----- 测试数据 -----
114 V testLines[][2] = {
115     {{220, 200}, {500, 380}}, // 完全在内
116     {{ 60, 480}, {120, 510}}, // 完全在外（同侧）
117     {{ 50, 280}, {680, 280}}, // 横跨左右两边
118     {{360, 50}, {360, 500}}, // 纵跨上下两边
119     {{100, 100}, {300, 250}}, // 对角穿过单个角
120     {{180, 250}, {180, 350}} // 共线于左边界
121 };
122 const int NL = sizeof(testLines) / sizeof(testLines[0]);
123
124 std::vector<V> testPolygon = {
125     {130, 100}, {520, 60}, {600, 200}, {500, 350},
126     {350, 480}, {300, 300}, {200, 460}, {120, 350}
127 };
128
129 // ----- 显示 -----
130 void display() {
131     glClear(GL_COLOR_BUFFER_BIT);
132     glLineWidth(1.5f);
133 }
```

```
134     drawClipWindow();
135
136     // 红色: 原始线段
137     glColor3f(1.0f, 0.3f, 0.3f);
138     for (int i = 0; i < NL; ++i) drawLine(testLines[i][0],
139         testLines[i][1]);
140     // 红色虚线: 原始多边形
141     glEnable(GL_LINE_STIPPLE);
142     glLineStipple(1, 0x0F0F);
143     drawPoly(testPolygon);
144     glDisable(GL_LINE_STIPPLE);
145
146     // 蓝色: 裁剪后的线段
147     glLineWidth(3.0f);
148     glColor3f(0.0f, 0.4f, 1.0f);
149     for (int i = 0; i < NL; ++i) {
150         V A = testLines[i][0], B = testLines[i][1];
151         if (cohenSutherland(A, B)) drawLine(A, B);
152     }
153     // 蓝色实线: 裁剪后的多边形
154     auto out = sutherlandHodgman(testPolygon);
155     drawPoly(out);
156
157     glFlush();
158 }
159
160 void keyboard(unsigned char key, int, int) {
161     double step = 10;
162     switch (key) {
163         case 'a': xL -= step; break;
164         case 'd': xL += step; break;
165         case 'w': yT += step; break;
166         case 's': yT -= step; break;
167         case 27 : std::exit(0);
168         default : return;
169     }
170     glutPostRedisplay();
171 }
172
173 void init() {
```

```

173     glClearColor(1, 1, 1, 1);
174     glEnable(GL_LINE_SMOOTH);
175     glHint(GL_LINE_SMOOTH_HINT, GL_NICEST);
176     glEnable(GL_BLEND);
177     glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
178     glMatrixMode(GL_PROJECTION);
179     gluOrtho2D(0, W, 0, H);
180 }
181
182 int main(int argc, char** argv) {
183     glutInit(&argc, argv);
184     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
185     glutInitWindowSize(W, H);
186     glutInitWindowPosition(100, 100);
187     glutCreateWindow("Exp.7 Clipping: Cohen-Sutherland +
        Sutherland-Hodgman");
188     init();
189     glutDisplayFunc(display);
190     glutKeyboardFunc(keyboard);
191     glutMainLoop();
192     return 0;
193 }

```

Listing 7: 直线 / 多边形裁剪——完整源代码

9.8 实验八代码

程序中立方体的 8 个原始顶点放在 $[-L/2, L/2]^3$ 区间，12 条边由编号对自动生成。每次按键事件构造对应的 4×4 矩阵，更新累积变换矩阵 M ；显示时对原始顶点先做模型变换、再做正等测投影、最后绘制 12 条边。

```

1  // =====
2  // 实验八：三维图形变换 —— 立方体 + 正等测投影
3  // 编译: cl /EHsc exp8.cpp freeglut.lib opengl32.lib glu32.lib
4  // =====
5  #include <GL/freeglut.h>
6  #include <cmath>
7  #include <cstdlib>
8
9  const int W = 720, H = 540;
10 const double PI = 3.14159265358979323846;

```

```
11  const double DEG = PI / 180.0;
12  const double SCREEN_OFFSET_X = 360, SCREEN_OFFSET_Y = 270;
13
14  // ----- 4x4 矩阵（行优先，行向量右乘） -----
15  struct M4 { double m[4][4]; };
16
17  M4 mIdentity() {
18      M4 r{};
19      for (int i = 0; i < 4; ++i) r.m[i][i] = 1;
20      return r;
21  }
22  M4 mMul(const M4& A, const M4& B) {
23      M4 r{};
24      for (int i = 0; i < 4; ++i)
25          for (int j = 0; j < 4; ++j)
26              for (int k = 0; k < 4; ++k)
27                  r.m[i][j] += A.m[i][k] * B.m[k][j];
28      return r;
29  }
30  M4 mTrans(double tx, double ty, double tz) {
31      M4 r = mIdentity();
32      r.m[3][0] = tx; r.m[3][1] = ty; r.m[3][2] = tz;
33      return r;
34  }
35  M4 mScale(double sx, double sy, double sz) {
36      M4 r = mIdentity();
37      r.m[0][0] = sx; r.m[1][1] = sy; r.m[2][2] = sz;
38      return r;
39  }
40  M4 mRotX(double deg) {
41      double t = deg * DEG, c = std::cos(t), s = std::sin(t);
42      M4 r = mIdentity();
43      r.m[1][1] = c; r.m[1][2] = s;
44      r.m[2][1] = -s; r.m[2][2] = c;
45      return r;
46  }
47  M4 mRotY(double deg) {
48      double t = deg * DEG, c = std::cos(t), s = std::sin(t);
49      M4 r = mIdentity();
50      r.m[0][0] = c; r.m[0][2] = -s;
```

```

51     r.m[2][0] = s; r.m[2][2] = c;
52     return r;
53 }
54 M4 mRotZ(double deg) {
55     double t = deg * DEG, c = std::cos(t), s = std::sin(t);
56     M4 r = mIdentity();
57     r.m[0][0] = c; r.m[0][1] = s;
58     r.m[1][0] = -s; r.m[1][1] = c;
59     return r;
60 }
61 M4 mReflectXY() { M4 r = mIdentity(); r.m[2][2] = -1; return r; }
62 M4 mReflectYZ() { M4 r = mIdentity(); r.m[0][0] = -1; return r; }
63 M4 mReflectXZ() { M4 r = mIdentity(); r.m[1][1] = -1; return r; }
64
65 // ----- 正等测投影: (x,y,z) -> (xs, ys) -----
66 void isoProject(double x, double y, double z, double& xs, double& ys) {
67     const double c30 = std::cos(30 * DEG), s30 = std::sin(30 * DEG);
68     xs = (x - y) * c30 + SCREEN_OFFSET_X;
69     ys = (x + y) * s30 - z + SCREEN_OFFSET_Y;
70 }
71
72 // ----- 立方体: 8 顶点 + 12 边 -----
73 const double L = 120;
74 double V0[8][3] = {
75     {-L/2, -L/2, -L/2}, { L/2, -L/2, -L/2}, { L/2,  L/2, -L/2}, {-L/2,  L/2, -L/2},
76     {-L/2, -L/2,  L/2}, { L/2, -L/2,  L/2}, { L/2,  L/2,  L/2}, {-L/2,  L/2,  L/2}
77 };
78 int E[12][2] = {
79     {0,1},{1,2},{2,3},{3,0}, // 底面
80     {4,5},{5,6},{6,7},{7,4}, // 顶面
81     {0,4},{1,5},{2,6},{3,7} // 立柱
82 };
83
84 M4 curM = mIdentity();
85 double cx = 0, cy = 0, cz = 0; // 立方体中心 (在物体本体坐标系中始终为 0)
86
87 // 三明治: T(-c) * Mb * T(+c); 这里 c 即当前累积平移分量 (curM 的第 4 行)
88 M4 sandwich(const M4& Mb) {
89     double tx = curM.m[3][0], ty = curM.m[3][1], tz = curM.m[3][2];
90     return mMul(mMul(mTrans(-tx, -ty, -tz), Mb), mTrans(tx, ty, tz));

```

```

91 }
92
93 // ----- 应用 4x4 变换:  $[x \ y \ z \ 1] * M$  -----
94 void applyM(const M4& M, double x, double y, double z,
95            double& xo, double& yo, double& zo) {
96     xo = x * M.m[0][0] + y * M.m[1][0] + z * M.m[2][0] + M.m[3][0];
97     yo = x * M.m[0][1] + y * M.m[1][1] + z * M.m[2][1] + M.m[3][1];
98     zo = x * M.m[0][2] + y * M.m[1][2] + z * M.m[2][2] + M.m[3][2];
99 }
100
101 // ----- 显示 -----
102 void drawCube(const M4& M, bool dashed) {
103     if (dashed) { glEnable(GL_LINE_STIPPLE); glLineStipple(1, 0x00FF); }
104     glBegin(GL_LINES);
105     for (int i = 0; i < 12; ++i) {
106         double x1, y1, z1, x2, y2, z2;
107         applyM(M, V0[E[i][0]][0], V0[E[i][0]][1], V0[E[i][0]][2], x1, y1,
108                z1);
109         applyM(M, V0[E[i][1]][0], V0[E[i][1]][1], V0[E[i][1]][2], x2, y2,
110                z2);
111         double xs1, ys1, xs2, ys2;
112         isoProject(x1, y1, z1, xs1, ys1);
113         isoProject(x2, y2, z2, xs2, ys2);
114         glVertex2d(xs1, ys1);
115         glVertex2d(xs2, ys2);
116     }
117     glEnd();
118     if (dashed) glDisable(GL_LINE_STIPPLE);
119 }
120
121 // 屏幕中心绘制三个等测投影下的坐标轴标记
122 void drawAxes() {
123     double Lx = 200;
124     glColor3f(0.5f, 0.5f, 0.5f);
125     glBegin(GL_LINES);
126     double xs, ys;
127     // X 轴
128     isoProject(0, 0, 0, xs, ys); glVertex2d(xs, ys);
129     isoProject(Lx, 0, 0, xs, ys); glVertex2d(xs, ys);
130     // Y 轴

```

```
129     isoProject(0, 0, 0, xs, ys); glVertex2d(xs, ys);
130     isoProject(0,Lx, 0, xs, ys); glVertex2d(xs, ys);
131     // Z 轴
132     isoProject(0, 0, 0, xs, ys); glVertex2d(xs, ys);
133     isoProject(0, 0,Lx, xs, ys); glVertex2d(xs, ys);
134     glEnd();
135 }
136
137 void display() {
138     glClear(GL_COLOR_BUFFER_BIT);
139     glLineWidth(1.5f);
140     drawAxes();
141
142     // 原始立方体（灰色虚线，单位变换）
143     glColor3f(0.6f, 0.6f, 0.6f);
144     drawCube(mIdentity(), true);
145
146     // 变换后立方体（红色实线）
147     glColor3f(1.0f, 0.0f, 0.0f);
148     glLineWidth(2.5f);
149     drawCube(curM, false);
150
151     glFlush();
152 }
153
154 // ----- 输入处理 -----
155 void apply(const M4& step, bool aroundCenter) {
156     if (aroundCenter) curM = mMul(curM, sandwich(step));
157     else curM = mMul(curM, step);
158     glutPostRedisplay();
159 }
160
161 void keyboard(unsigned char key, int, int) {
162     switch (key) {
163         case '1': apply(mRotX( 5), true); break;
164         case '!': apply(mRotX(-5), true); break;
165         case '2': apply(mRotY( 5), true); break;
166         case '@': apply(mRotY(-5), true); break;
167         case '3': apply(mRotZ( 5), true); break;
168         case '#': apply(mRotZ(-5), true); break;
```

```
169     case '+': case '=': apply(mScale(1.05, 1.05, 1.05), true); break;
170     case '-': case '_': apply(mScale(1/1.05, 1/1.05, 1/1.05), true);
        break;
171     case 'x': apply(mReflectYZ(), true); break;
172     case 'y': apply(mReflectXZ(), true); break;
173     case 'z': apply(mReflectXY(), true); break;
174     case 'Z': curM = mIdentity(); break;
175     case 27 : std::exit(0);
176 }
177 glutPostRedisplay();
178 }
179
180 void special(int key, int, int) {
181     double step = 20;
182     switch (key) {
183         case GLUT_KEY_LEFT: apply(mTrans(-step, 0, 0), false); break;
184         case GLUT_KEY_RIGHT: apply(mTrans( step, 0, 0), false); break;
185         case GLUT_KEY_DOWN: apply(mTrans(0,-step, 0), false); break;
186         case GLUT_KEY_UP: apply(mTrans(0, step, 0), false); break;
187         case GLUT_KEY_PAGE_UP: apply(mTrans(0,0, step), false); break;
188         case GLUT_KEY_PAGE_DOWN: apply(mTrans(0,0,-step), false); break;
189     }
190 }
191
192 void init() {
193     glClearColor(1, 1, 1, 1);
194     glEnable(GL_LINE_SMOOTH);
195     glHint(GL_LINE_SMOOTH_HINT, GL_NICEST);
196     glEnable(GL_BLEND);
197     glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
198     glMatrixMode(GL_PROJECTION);
199     gluOrtho2D(0, W, 0, H);
200 }
201
202 int main(int argc, char** argv) {
203     glutInit(&argc, argv);
204     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
205     glutInitWindowSize(W, H);
206     glutInitWindowPosition(100, 100);
207     glutCreateWindow("Exp.8 3D Transformations + Isometric Projection");
```

```
208     init();
209     glutDisplayFunc(display);
210     glutKeyboardFunc(keyboard);
211     glutSpecialFunc(special);
212     glutMainLoop();
213     return 0;
214 }
```

Listing 8: 三维图形变换 + 正等测投影——完整源代码